

ENGINEERING GUIDE

PharmaChain

From a DBMS Coursework Project to a
Production-Grade Spring Boot Platform

[A Complete Architecture, Security, AI, and DevOps Guide](#)

Documenting every layer added on top of a PostgreSQL
pharmaceutical-manufacturing schema — what was built, why it was built
that way, and every defect found and corrected along the way.

Subsystems documented: PostgreSQL schema, triggers, views & procedures
Spring Boot REST + JPA
Spring Security (JWT, RBAC)
Spring AI (RAG + tool-calling)
Docker, Compose & CI/CD
Testcontainers integration testing
Full architectural code review

Abstract

This guide documents the complete transformation of PharmaChain from a self-contained database coursework project into a layered, secured, AI-augmented, containerized backend platform. The starting point was a PostgreSQL schema modelling a pharmaceutical manufacturing operation: nineteen normalized tables, four compliance-enforcing triggers, three reporting views, and one emergency-recall stored procedure. Nothing about that starting point was thrown away; the guiding principle throughout this work was that the database already contained real, hard-won business logic, and the job of every layer added on top was to *expose* that logic through a modern API, never to quietly re-implement (and thereby risk diverging from) it.

Five upgrade phases are documented in full, each as a self-contained chapter set with working code: a Spring Data JPA persistence layer mapped directly onto the existing schema; a stateless JWT authentication and role-based authorization layer; a Spring AI compliance copilot combining retrieval-augmented generation with tool-calling; a multi-stage Docker build with a full `docker-compose` development stack and a GitHub Actions CI/CD pipeline; and a Testcontainers-based integration test suite that exercises the real PostgreSQL triggers rather than an in-memory substitute that could not understand them.

The final chapters depart from a typical build narrative and document a formal architectural review conducted after the platform was feature-complete. Sixteen concrete defects were found and corrected, the most significant being a systemic data-integrity bug affecting the majority of the write API: Spring Data JPA's `save()` silently updates an existing row instead of failing whenever an entity's primary key is client-supplied and already in use, because it internally calls `merge()` rather than `persist()` once the identifier is non-null. This guide treats that discovery, and the fifteen others alongside it, with the same level of technical depth as the original feature build — explaining not only what was wrong, but why it was wrong, how it was found, and why the chosen fix was preferred over the alternatives that were considered and rejected.

Every code listing in this document is drawn directly from the shipped source tree, not written for illustration after the fact. Where a design decision involved a real trade-off — Anthropic for chat but a local Ollama model for embeddings, entities without JPA object relationships, keeping the vector store in PostgreSQL's default schema rather than the project's own — the trade-off and the reasoning behind it are documented explicitly, so that a reader can disagree with a decision on its merits rather than have to reverse-engineer why it was made.

Contents

Abstract	i
Contents	iii
List of Figures	vii
List of Tables	ix
1 Introduction	1
1.1 Where This Project Started	1
1.2 Why Upgrade to Spring Boot	1
1.3 A Guiding Principle: The Database Is Not Legacy	2
1.4 How This Guide Is Organized	3
2 The Database Foundation	5
2.1 Schema Overview	5
2.1.1 An Identifier-Case Gotcha Worth Flagging Early	5
2.2 The Four Triggers	6
2.2.1 Trigger 1 — Stock Deduction on Dispensing	6
2.2.2 Trigger 2 — No Sale Without a Passed QC Result	6
2.2.3 Trigger 3 — Batch Date Sanity Checks	7
2.2.4 Trigger 4 — The Tamper-Evident QC Audit Trail	8
2.3 The Three Reporting Views	9
2.3.1 v_fda_batch_traceability	9
2.3.2 v_inventory_shortage	9
2.3.3 v_inventory_expiry_risk	10
2.4 The Emergency Recall Procedure	10
2.5 Summary	11
3 Architecture Overview	13
3.1 The Layered Architecture	13
3.2 Package Structure	13
3.3 Build Configuration	14
3.4 The Recurring Design Philosophy	15
4 The Persistence Layer	17
4.1 Entities Map Columns, Not Relationships	17
4.2 The PostgreSQL Identifier-Folding Trap	17
4.3 Composite Keys via @EmbeddedId	18
4.4 The Repository Layer	19
4.5 Entity Inventory	20
5 DTOs and Validation Strategy	23
5.1 Two Kinds of Request Body	23
5.2 Records as Request DTOs	23
5.3 Validation Gaps Are a Real Bug Class	24

5.4	Cross-Field Validation	25
5.5	Response DTOs	25
6	The Service Layer	27
6.1	Two Categories of Service	27
6.2	A Complete Worked Example: <code>BatchService</code>	27
6.3	JdbcTemplate for What JPA Doesn't Cover	29
6.3.1	Reading the Views	29
6.3.2	Calling the Stored Procedure	29
6.4	Transaction Boundaries	29
6.5	The Duplicate-ID Guard Pattern	30
7	The REST API and Exception Handling	31
7.1	Controller Conventions	31
7.2	The <code>ApiError</code> Contract	31
7.3	<code>GlobalExceptionHandler</code> : One Handler per Failure Mode	32
7.3.1	Unwrapping a Trigger's Message from <code>DataAccessException</code>	32
7.3.2	The <code>@PreAuthorize</code> Ordering Surprise	33
7.3.3	Client-Input Edge Cases	33
8	Authentication and Authorization	35
8.1	Design Goals	35
8.2	A New Table, Deliberately Separate from <code>Employee_Master</code>	35
8.3	Issuing and Validating Tokens	35
8.4	The Authentication Filter	36
8.5	Loading Users	37
8.6	The Filter Chain Configuration	38
8.6.1	A Genuine CORS Subtlety	39
8.7	Role-Based Access Control on Every Write Endpoint	39
8.8	Login	40
8.9	Demo Accounts	41
9	The Spring AI Compliance Copilot	43
9.1	What Was Built	43
9.2	A Deliberate Provider Split	43
9.3	Tool-Calling Instead of Raw SQL Access	44
9.4	Retrieval-Augmented Generation	45
9.4.1	The Knowledge Base	45
9.4.2	Ingestion	45
9.4.3	A Schema-Placement Decision That Prevented a Startup-Ordering Bug	46
9.4.4	Combining Retrieval and Tools in One Call	47
9.5	Structured Output: The Recall Notice Generator	47
9.6	Endpoint Summary	49
10	Containerization and CI/CD	51
10.1	A Multi-Stage Dockerfile	51
10.2	Completing the Health Check: Adding Actuator	52
10.3	Local Development: <code>docker-compose.yml</code>	52
10.4	The CI/CD Pipeline	53
10.5	Summary	55

11 Testing Strategy	57
11.1 Why an In-Memory Database Cannot Be Used Here	57
11.2 Loading the Real Schema Correctly	57
11.3 An Honest Constraint on This Entire Project	58
11.4 Test Inventory	58
11.5 A Representative Trigger Test	59
11.6 A Representative Regression Test	60
12 The Architecture Review	61
12.1 Methodology	61
12.2 The Central Finding: <code>save()</code> Silently Corrupts Data on a Duplicate Client-Supplied ID	61
12.2.1 The Mechanism	61
12.2.2 Why This Is Worse Than It Sounds for Composite Keys	62
12.2.3 The Fix, and the Alternative That Was Rejected	62
12.3 Every Other Finding	63
12.3.1 The Complete Validation-Gap Audit	64
12.3.2 Verifying Test Data Against Real Seed Data, Not Assumptions	64
12.4 Summary of All Findings	65
12.5 A Tooling Self-Correction, Told in Full	65
12.6 Verified Correct: What This Review Did <i>Not</i> Find Wrong	66
12.7 Recommendations, Deliberately Not Implemented	67
13 Retrospective	69
13.1 What Actually Changed	69
13.2 From Coursework to Something Closer to Production	69
13.3 What Would Change With More Time	70
13.4 Closing	70
A Reference Material	71
A.1 Complete API Reference	71
A.2 Demo Accounts	71
A.3 Environment Setup Quick Reference	71
A.4 Glossary	72

List of Figures

- 3.1 Request flow through PharmaChain's layers. The service layer is deliberately the only layer that talks to both the persistence subsystem and the AI subsystem — controllers never call `JdbcTemplate` or `ChatClient` directly. 13

List of Tables

1.1	Curriculum concepts and where each is applied in PharmaChain	2
2.1	The nineteen tables, grouped by function	5
2.2	Database objects and where they surface in the Spring Boot layer	11
3.1	Package structure and file counts	14
4.1	All 24 entity classes and their identifier strategy	21
5.1	A sample of validation gaps closed during the architecture review (full list in Chapter 12)	25
7.1	Every exception type handled, and what it means	32
8.1	Role-based write access, by resource	40
9.1	The two AI endpoints	49
10.1	Runtime environment variables	55
11.1	Every test method and what it proves	59
12.1	Every method patched against the <code>merge()</code> vs. <code>persist()</code> defect	64
12.2	All thirteen validation gaps closed, matched against their exact database constraint	65
12.3	All sixteen findings from the architecture review	66
13.1	From a database coursework project to a reviewed, deployable platform	69
A.1	Authentication	71
A.2	Master data (plain CRUD - GET open to any role on every resource below)	71
A.3	Production workflow	72
A.4	Commerce, AI, and dashboards	73
A.5	Seeded accounts for local development and grading	73

Chapter 1

Introduction

1.1 Where This Project Started

PharmaChain-DB began as a pure database-systems project: a PostgreSQL schema modelling the core operations of a pharmaceutical manufacturer, from raw-material procurement through production, quality control, sale, and — when something goes wrong — emergency recall. It is, by the standards of a coursework project, unusually complete. Most student database projects stop at a normalized schema with a handful of foreign keys. This one goes further: it encodes real regulatory and operational logic directly into the database as triggers, views, and a stored procedure, so that the rules governing the business are enforced by PostgreSQL itself and cannot be bypassed by a careless or buggy client.

Four rules in particular define the character of the whole system, and they recur throughout this guide because every later architectural decision was made in service of preserving them:

1. Raw material cannot be dispensed to production faster than it physically exists in the warehouse.
2. A batch of finished product cannot be sold until it has been tested and has *passed* quality control — no exceptions, no matter which application or client is making the request.
3. A batch's manufacturing date cannot be in the future, and its shelf life cannot be implausibly short.
4. Every change to a quality-control result is captured in a tamper-evident audit trail, automatically, without any application code having to remember to log it.

These four rules are the closest thing this project has to a mission statement. Section 2.2 documents each of them in full.

1.2 Why Upgrade to Spring Boot

A finished database schema is a static artifact: it can be queried with `psql`, inspected with a GUI client, and demonstrated with a slide of an entity-relationship diagram, but it cannot be handed to a frontend, called from a mobile app, or deployed as a service other systems depend on. The motivation for this upgrade was to turn that static schema into something that behaves like real infrastructure: a versioned, testable, independently deployable HTTP API, with the same guarantees the database already provided, plus the guarantees an API layer is expected to add on top — authentication, authorization, input validation, structured error responses, and observability.

The upgrade was undertaken as a deliberate curriculum capstone, applying a specific, ordered set of backend engineering concepts to a single, coherent domain rather than to a series of disjoint toy exercises. Table 1.1 lists every concept area covered and where it surfaces in this guide.

Table 1.1: Curriculum concepts and where each is applied in PharmaChain

Concept area	Applied as	Chapter
Core & Advanced Java	Records, streams, builder pattern, enums, exceptions	4–6
Maven	Multi-dependency build, BOM imports, plugin configuration	3
JDBC	Raw <code>JdbcTemplate</code> for views, procedures & AI ingestion	6, 9
Hibernate / JPA	Entity mapping, composite keys, transaction boundaries	4
Spring MVC & REST	Controllers, DTOs, status codes, content negotiation	7
Spring Data JPA	Repositories, derived queries, the <code>save()</code> pitfall	4, 12
Spring AOP	Method-level <code>@PreAuthorize</code> security interception	8
Spring Security	JWT, stateless sessions, role-based access control	8
JWT & OAuth2 concepts	Token issuance, validation, claims, expiry	8
Docker	Multi-stage builds, health checks, non-root runtime	10
Cloud deployment	Container registries, deploy hooks, CI/CD	10
Spring AI	RAG, tool-calling, structured output	9
Git & CI/CD	GitHub Actions build/test/publish pipeline	10
Testing & QA	Testcontainers, regression tests, formal code review	11, 12

1.3 A Guiding Principle: The Database Is Not Legacy

It would have been possible to treat the existing schema as a constraint to be worked around — to pull data out of it, re-implement every rule in Java, and treat PostgreSQL as a dumb row store. That approach was deliberately rejected. The triggers, views, and procedure already documented in PharmaChain-DB represent real engineering effort and, more importantly, they are the only place in the whole system where a rule is enforced *no matter what calls into the database* — not just when the call happens to go through this particular Spring Boot application.

WHY THIS MATTERS

A service layer that validates “the manufacturing date is not in the future” only protects callers of that specific service method. A database trigger enforcing the same rule protects every caller, forever: this application, a future microservice, an analyst running an ad-hoc `UPDATE` in `psql`, or a bulk-import script nobody has written yet. Throughout this guide, the phrase *defense in depth* refers specifically to this pattern: validate early in the application for a fast, friendly error message, but never remove or bypass the database-level enforcement of the same rule.

Every chapter that follows a piece of business logic — dispensing material, submitting a QC result, recording a sale — shows this pattern in practice: an application-level check that fails fast with a clear message, sitting directly above a database constraint or trigger that would have caught the same problem regardless.

1.4 How This Guide Is Organized

The chapters are ordered the way the platform was actually built, phase by phase, so that later decisions can be understood in the context of what already existed when they were made:

- **Chapter 2** documents the database foundation as it already existed: the schema, the four triggers, the three views, and the recall procedure, with the reasoning behind each.
- **Chapters 3–7** document the Spring Boot core: the layered architecture, how the schema was mapped to JPA entities (including a genuine PostgreSQL identifier-folding pitfall), the DTO and validation strategy, the service layer’s defense-in-depth pattern, and the REST/exception-handling layer.
- **Chapter 8** documents the JWT authentication and role-based authorization layer, including two real security defects found during later review and how they were fixed.
- **Chapter 9** documents the Spring AI compliance copilot: a retrieval-augmented question-answering system combined with tool-calling over live operational data, plus a structured-output recall-notice generator.
- **Chapter 10** documents containerization, the local development stack, and the CI/CD pipeline.
- **Chapter 11** documents the integration testing strategy, built around Testcontainers specifically because an in-memory database cannot understand PL/pgSQL triggers.
- **Chapter 12** is the most unusual chapter in this guide: a full architectural review conducted after the platform was considered feature-complete, documenting sixteen real defects found and fixed, several of them non-obvious enough to be worth a deep, standalone explanation.
- **Chapter 13** closes with a retrospective and a list of deliberately-out-of-scope recommendations for future work.

Chapter 2

The Database Foundation

Before any Java is written, it is worth understanding exactly what the database already guarantees, because every later chapter in this guide is, in one way or another, about exposing these guarantees through an API without weakening them.

2.1 Schema Overview

The schema lives entirely inside a dedicated `pharma_manufacturing` PostgreSQL schema and comprises nineteen tables, which fall naturally into five functional groups.

Table 2.1: The nineteen tables, grouped by function

Group	Tables
Master data	Material_Master, Account_Master, Employee_Master, Equipment_Master, Product_Master
Commerce & contracts	Transactions, Supplier_Contract, RM_Transaction, FG_Transaction
Inventory	Warehouse, Material_Quality_Check
Production	Formula_Master, Batch, Material_Dispensing, Production_Log
Post-production & compliance	Product_Quality_Check, FG_Transaction, Product_Recall, Maintenance_Log, QC_Audit_Log

`Transactions` deserves a specific note: it is a single, unified ledger table for *both* raw-material purchases (`Transaction_Type = 'buy'`) and finished-goods sales (`Transaction_Type = 'sell'`), with `RM_Transaction` and `FG_Transaction` each hanging off it via `Invoice_No` as the line-item detail for their respective direction. This detail matters later: [Chapter 12](#) documents a defect where a duplicate `Invoice_No` could silently corrupt an *unrelated* transaction precisely because both buy and sell share this one table.

2.1.1 An Identifier-Case Gotcha Worth Flagging Early

The DDL is written with unquoted, mixed-case identifiers throughout — `Material_Master`, `Material_ID`, and so on. PostgreSQL folds every unquoted identifier to lowercase at creation time, so the table actually stored on disk is `material_master`, with a column named `material_id`, not `Material_ID`. This is invisible in `psql` (which displays column names case-insensitively by default) and invisible in the DDL itself, but it is not invisible to Hibernate, which maps Java fields to *exact* column names. [Chapter 4](#) documents this in full, because getting it wrong produces a runtime error only when the application actually tries to talk to the database — exactly the kind of mismatch that is easy to introduce and painful to debug.

2.2 The Four Triggers

2.2.1 Trigger 1 — Stock Deduction on Dispensing

```
CREATE OR REPLACE FUNCTION automate_warehouse_stock()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_current_stock NUMERIC(10);
BEGIN
    SELECT Stock INTO v_current_stock
    FROM Warehouse
    WHERE Item_ID = NEW.Item_ID;

    IF v_current_stock < NEW.Quantity_Issued THEN
        RAISE EXCEPTION 'Transaction Blocked: Not enough stock! Item %
            only has % units left, but you tried to dispense %.',
            NEW.Item_ID, v_current_stock, NEW.Quantity_Issued;
    END IF;

    UPDATE Warehouse
    SET Stock = Stock - NEW.Quantity_Issued
    WHERE Item_ID = NEW.Item_ID;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_deduct_stock_on_dispense
AFTER INSERT ON Material_Dispensing
FOR EACH ROW
EXECUTE FUNCTION automate_warehouse_stock();
```

Listing 2.1: `automate_warehouse_stock()`, bound AFTER INSERT on `Material_Dispensing`

Every raw-material dispensing event is checked against the warehouse lot's live stock and, if sufficient, atomically decrements it in the same transaction as the insert. If the request would overdraw the lot, the exception aborts the entire statement — nothing is written, on either table.

WHY THIS MATTERS

This is a textbook overselling problem, generalized from finished-goods inventory (where most engineers first encounter it) to raw materials. Two concurrent dispensing requests against the same lot cannot both succeed if there is only enough stock for one; enforcing the check and the deduction inside a single trigger, rather than as a read-then-write in application code, closes the race condition that a “check in Java, then write to the database” approach would leave open.

2.2.2 Trigger 2 — No Sale Without a Passed QC Result

```
CREATE OR REPLACE FUNCTION enforce_quality_control()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
```

```

DECLARE
    v_qa_result VARCHAR(30);
BEGIN
    SELECT Results INTO v_qa_result
    FROM Product_Quality_Check
    WHERE Batch_No = NEW.Batch_No;

    IF v_qa_result IS NULL THEN
        RAISE EXCEPTION 'COMPLIANCE BLOCK: Batch % has not been tested
            by the QC Lab yet. Sale is illegal!', NEW.Batch_No;
    END IF;

    IF v_qa_result = 'FAILED' THEN
        RAISE EXCEPTION 'SAFETY BLOCK: Batch % FAILED quality control.
            This batch must be quarantined, not sold!', NEW.Batch_No;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_prevent_bad_sales
BEFORE INSERT ON FG_Transaction
FOR EACH ROW
EXECUTE FUNCTION enforce_quality_control();

```

Listing 2.2: `enforce_quality_control()`, bound BEFORE INSERT on FG_Transaction

This is the trigger that most directly earns PharmaChain the right to call itself a compliance system rather than just an inventory system. A batch with no `Product_Quality_Check` row at all is treated identically to one that failed outright: both block the sale.

WHY THIS MATTERS

In a real pharmaceutical manufacturer, shipping an untested or failed batch is not a business mistake, it is a regulatory and patient-safety failure. Enforcing this as a BEFORE INSERT trigger — rather than only as an application-level check — means the rule holds even against a direct `psql` session, a future microservice, or a bulk-import script that bypasses the Spring Boot API entirely. Section 8.7 in Chapter 8 shows the analogous application-level check that exists *alongside* this trigger, not instead of it.

2.2.3 Trigger 3 — Batch Date Sanity Checks

```

CREATE OR REPLACE FUNCTION enforce_batch_dates()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.Mfg_Date > CURRENT_DATE THEN
        RAISE EXCEPTION 'TIME TRAVEL DETECTED: You cannot set a
            Manufacturing Date (%) in the future!', NEW.Mfg_Date;
    END IF;

    IF NEW.Exp_Date < (NEW.Mfg_Date + INTERVAL '6 months') THEN
        RAISE EXCEPTION 'QUALITY BLOCK: The Expiry Date (%) must be at
            least 6 months after the Manufacturing Date (%)',
            NEW.Exp_Date, NEW.Mfg_Date;
    END IF;
END;

```

```

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_strict_batch_dates
BEFORE INSERT OR UPDATE ON Batch
FOR EACH ROW
EXECUTE FUNCTION enforce_batch_dates();

```

Listing 2.3: `enforce_batch_dates()`, bound BEFORE INSERT OR UPDATE on Batch

Two independent checks, both catching data-entry errors before they corrupt downstream traceability: a manufacturing date cannot be in the future, and the gap between manufacturing and expiry must be at least six months. Because this trigger fires on UPDATE as well as INSERT, correcting an existing batch’s dates is held to the same standard as creating a new one.

2.2.4 Trigger 4 — The Tamper-Evident QC Audit Trail

```

CREATE OR REPLACE FUNCTION track_qc_changes()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        IF OLD.Results IS DISTINCT FROM NEW.Results THEN
            INSERT INTO QC_Audit_Log
                (Report_ID, Action_Type, Old_Result, New_Result, Changed_By)
            VALUES
                (OLD.Report_ID, 'UPDATE', OLD.Results, NEW.Results,
            CURRENT_USER);
        END IF;
        RETURN NEW;

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO QC_Audit_Log
            (Report_ID, Action_Type, Old_Result, New_Result, Changed_By)
        VALUES
            (OLD.Report_ID, 'DELETE', OLD.Results, 'RECORD DESTROYED',
        CURRENT_USER);
        RETURN OLD;
    END IF;
END;
$$;

CREATE TRIGGER trg_audit_qc_changes
AFTER UPDATE OR DELETE ON Product_Quality_Check
FOR EACH ROW
EXECUTE FUNCTION track_qc_changes();

```

Listing 2.4: `track_qc_changes()`, bound AFTER UPDATE OR DELETE on Product_Quality_Check

Nothing in the application is ever expected to insert into `QC_Audit_Log` directly — only this trigger does. That is the entire point of the design: because the audit entry is generated by the database itself as an automatic side effect of the change, rather than by application code that has to remember to log it, there is no code path capable of modifying a QC result without leaving a trace.

COURSE CONCEPT APPLIED

This mirrors the spirit (though not the full letter) of FDA 21 CFR Part 11's requirement for secure, computer-generated, time-stamped audit trails on electronic records in regulated industries. Chapter 9's compliance-copilot knowledge base includes a document explaining this connection in more depth, so that the AI system can answer questions about *why* the audit trail exists, not just recite that it does.

2.3 The Three Reporting Views

Three read-only views exist specifically so that the joins and aggregation logic behind common operational questions live in exactly one place — the database — instead of being re-implemented, and potentially re-implemented *inconsistently*, by every report or dashboard that needs the answer.

2.3.1 v_fda_batch_traceability

```
CREATE OR REPLACE VIEW v_fda_batch_traceability AS
SELECT
    B.Batch_No,
    PM.Product_Name,
    B.Mfg_Date,
    B.Exp_Date,
    COALESCE(QC.Results, 'UNTESTED') AS QC_Status,
    STRING_AGG(DISTINCT MM.Material_Name, ', ') AS Raw_Materials_Used,
    COALESCE(SUM(FGT.Sale_Qty), 0) AS Total_Sold_To_Market
FROM Batch B
JOIN Product_Master PM ON B.Product_ID = PM.Product_ID
LEFT JOIN Product_Quality_Check QC ON B.Batch_No = QC.Batch_No
LEFT JOIN Material_Dispensing MD ON B.Batch_No = MD.Batch_No
LEFT JOIN Warehouse W ON MD.Item_ID = W.Item_ID
LEFT JOIN Material_Master MM ON W.Material_ID = MM.Material_ID
LEFT JOIN FG_Transaction FGT ON B.Batch_No = FGT.Batch_No
GROUP BY B.Batch_No, PM.Product_Name, B.Mfg_Date, B.Exp_Date, QC.Results;
```

Listing 2.5: Full lifecycle record for one batch

Answers, in one query, the question a regulator or internal auditor would actually ask: given a batch number, what product is it, what is its current QC status, which raw materials (traced back through the warehouse lots that were dispensed into it) went into it, and how much has been sold into the market so far. This is exactly the query a recall-impact investigation starts from.

2.3.2 v_inventory_shortage

```
CREATE OR REPLACE VIEW v_inventory_shortage AS
SELECT
    w.Item_ID, m.Material_Name, m.Material_Type,
    w.Stock AS Current_Stock,
    m.Reorder_Level AS Minimum_Required,
    (m.Reorder_Level - w.Stock) AS Units_To_Order
FROM Warehouse w
JOIN Material_Master m ON w.Material_ID = m.Material_ID
WHERE w.Stock <= m.Reorder_Level;
```

Listing 2.6: Materials at or below their reorder point

A direct answer to “what do we need to reorder right now,” including exactly how many units the gap represents.

2.3.3 v_inventory_expiry_risk

```
CREATE OR REPLACE VIEW v_inventory_expiry_risk AS
SELECT
    B.Batch_No, PM.Product_Name, B.Exp_Date,
    (B.Exp_Date - CURRENT_DATE) AS Days_Remaining,
    CASE
        WHEN (B.Exp_Date - CURRENT_DATE) < 0 THEN 'EXPIRED - DO NOT SELL'
        WHEN (B.Exp_Date - CURRENT_DATE) <= 30 THEN 'CRITICAL - UNDER 30 DAYS'
        WHEN (B.Exp_Date - CURRENT_DATE) <= 90 THEN 'WARNING - UNDER 90 DAYS'
        ELSE 'SAFE'
    END AS Risk_Status,
    B.Stock_Qty AS Manufactured_Qty,
    COALESCE(SUM(FGT.Sale_Qty), 0) AS Total_Sold_Qty,
    (B.Stock_Qty - COALESCE(SUM(FGT.Sale_Qty), 0)) AS Unsold_Inventory
FROM Batch B
JOIN Product_Master PM ON B.Product_ID = PM.Product_ID
LEFT JOIN FG_Transaction FGT ON B.Batch_No = FGT.Batch_No
GROUP BY B.Batch_No, PM.Product_Name, B.Exp_Date, B.Stock_Qty
HAVING (B.Stock_Qty - COALESCE(SUM(FGT.Sale_Qty), 0)) > 0
ORDER BY Days_Remaining ASC;
```

Listing 2.7: Unsold stock ranked by proximity to expiry

Note the HAVING clause: fully-sold batches are excluded entirely, so this view only ever shows inventory that genuinely still needs a decision made about it, pre-sorted by urgency.

2.4 The Emergency Recall Procedure

```
CREATE OR REPLACE PROCEDURE execute_product_recall(
    p_recall_id VARCHAR(20),
    p_batch_no NUMERIC,
    p_reason VARCHAR(255)
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_unsold_qty NUMERIC;
BEGIN
    SELECT Stock_Qty INTO v_unsold_qty FROM Batch WHERE Batch_No = p_batch_no;

    IF v_unsold_qty IS NULL THEN
        RAISE EXCEPTION 'RECALL FAILED: Batch % does not exist.', p_batch_no;
    END IF;
    IF v_unsold_qty <= 0 THEN
        RAISE EXCEPTION 'RECALL FAILED: Batch % has 0 stock. Nothing to quarantine!', p_batch_no;
    END IF;

    INSERT INTO Product_Recall (Recall_ID, Batch_No, Date_Initiated, Reason, Qty_Recalled)
    VALUES (p_recall_id, p_batch_no, CURRENT_TIMESTAMP, p_reason, v_unsold_qty);
END;
```

```

VALUES (p_recall_id, p_batch_no, CURRENT_DATE, p_reason, v_unsold_qty);

UPDATE Batch SET Stock_Qty = 0 WHERE Batch_No = p_batch_no;

UPDATE Product_Quality_Check SET Results = 'RECALLED' WHERE Batch_No =
p_batch_no;
END;
$$;

```

Listing 2.8: execute_product_recall: three effects, one atomic operation

A single call performs three effects atomically: it logs the recall, zeroes the batch's saleable stock, and quarantines it by overwriting its QC result to `RECALLED`. That third effect is what makes the recall permanent from the sales side too — because `trg_prevent_bad_sales` only allows selling a batch whose QC result is `PASSED`, quarantining and blocking future sales reuse the exact same mechanism, rather than needing a second, separate “is this batch recalled” check anywhere else in the system.

DESIGN DECISION

Chapter 6 shows `RecallService` calling this procedure directly via a raw JDBC `CallableStatement`, rather than re-implementing its three steps in Java. Spring Data JPA has no first-class support for PostgreSQL PROCEDURES (only FUNCTIONS via `@Procedure`), and re-implementing the three steps in application code would have reintroduced exactly the kind of divergence-risk this whole guide's design philosophy exists to avoid.

2.5 Summary

Table 2.2 consolidates every trigger, view, and procedure documented in this chapter, alongside the chapter of this guide where its Java-side integration is covered.

Table 2.2: Database objects and where they surface in the Spring Boot layer

Object	Fires on	Java-side integration
<code>trg_deduct_stock_on_dispense</code>	<code>Material_Dispensing</code> , AFTER INSERT	<code>BatchService.dispenseMaterial()</code>
<code>trg_prevent_bad_sales</code>	<code>FG_Transaction</code> , BEFORE INSERT	<code>SalesService.recordSale()</code>
<code>trg_strict_batch_dates</code>	<code>Batch</code> , BEFORE INS./UPD.	<code>BatchService.createBatch()</code>
<code>trg_audit_qc_changes</code>	<code>Product_Quality_Check</code> , AFTER UPD./DEL.	QC audit-trail endpoint
<code>v_fda_batch_traceability</code>	— (view)	<code>DashboardService.traceability()</code>
<code>v_inventory_shortage</code>	— (view)	<code>DashboardService.inventoryShortage()</code>
<code>v_inventory_expiry_risk</code>	— (view)	<code>DashboardService.expiryRisk()</code>
<code>execute_product_recall</code>	— (procedure)	<code>RecallService.initiateRecall()</code>

Architecture Overview

3.1 The Layered Architecture

PharmaChain’s backend follows a conventional layered architecture, deliberately kept conventional: the interesting engineering decisions in this project are about *what each layer is responsible for* relative to the database, not about inventing a novel structure. Figure 3.1 shows the full picture, including where the two newer subsystems — security and AI — sit relative to the original request/response path.

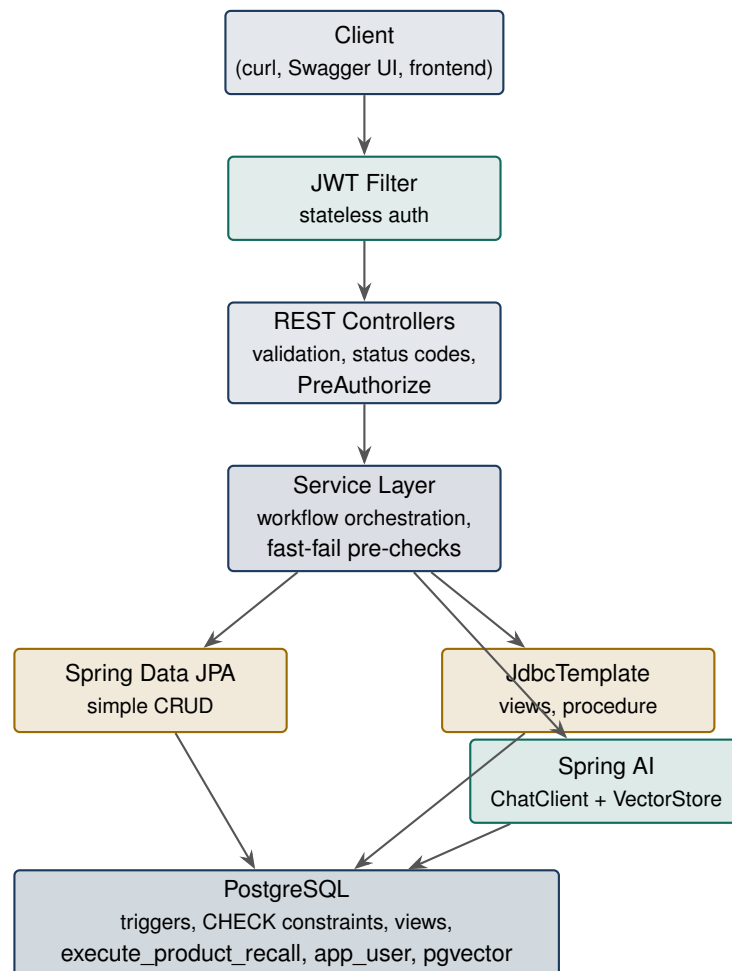


Figure 3.1: Request flow through PharmaChain’s layers. The service layer is deliberately the only layer that talks to both the persistence subsystem and the AI subsystem — controllers never call `JdbcTemplate` or `ChatClient` directly.

3.2 Package Structure

The application is organized by technical layer rather than by feature/domain, which is a deliberate choice appropriate to this project’s size: at nineteen underlying tables and roughly

a hundred Java files, a layer-first package structure keeps related concerns (every entity, every repository, every controller) easy to scan as a group, without the added indirection a feature-first structure would introduce for a codebase this size.

Table 3.1: Package structure and file counts

Package	Responsibility	Files
entity	JPA entities, one per table, plus four @Embeddable composite-key classes	24
repository	JpaRepository interfaces, one per entity	20
dto.request / dto.response	Records for workflow request/response bodies	16
service	Business logic, transaction boundaries, defense-in-depth checks	15
controller	REST endpoints, HTTP status codes, @PreAuthorize	16
security	JWT issuance/validation, filter, UserDetails Service, config	6
ai	Spring AI configuration, tool-calling, RAG, structured output	6
exception	ApiError, custom exceptions, the global handler	4
config	Cross-cutting Spring configuration (OpenAPI metadata)	1

3.3 Build Configuration

The project builds with Maven against Spring Boot 3.3.4 and Java 21. Three Bills of Materials (BOMs) are imported to keep transitive dependency versions consistent without pinning each library by hand: `spring-boot-starter-parent` (implicit, as the project's parent POM), `testcontainers-bom 1.20.1`, and `spring-ai-bom 1.0.1`.

```
<!-- Core web + data -->
spring-boot-starter-web
spring-boot-starter-actuator
spring-boot-starter-data-jpa
spring-boot-starter-validation
postgresql                                <!-- NOT runtime-scoped: see Chapter 7 -->
springdoc-openapi-starter-webmvc-ui      <!-- Swagger UI -->
lombok

<!-- Security -->
spring-boot-starter-security
jjwt-api / jjwt-impl / jjwt-jackson    <!-- v0.13.0 -->

<!-- Spring AI -->
spring-ai-starter-model-anthropic
spring-ai-starter-model-ollama
spring-ai-starter-vector-store-pgvector

<!-- Testing -->
spring-boot-starter-test
spring-security-test
testcontainers (junit-jupiter, postgresql)
```

Listing 3.1: Core dependency groups in `pom.xml`**DESIGN DECISION**

The `postgresql` JDBC driver is deliberately *not* scoped as `runtime` even though that is the conventional scoping for a JDBC driver. Chapter 7's `GlobalExceptionHandler` references `org.postgresql.util.PSQLException` directly at compile time, to unwrap the human-readable message a database trigger raised via `RAISE EXCEPTION`. Scoping the driver as `runtime` would have made the project fail to compile the moment that exception-handling code was written — a small, easy-to-miss trap for anyone copying a “standard” Spring Boot `pom.xml` template without reading it critically.

3.4 The Recurring Design Philosophy

Three principles, introduced in Chapter 1 and restated here because they apply mechanically at every layer described in the chapters that follow:

1. **Defense in depth.** A business rule already enforced by a trigger or constraint is re-checked in the service layer only for a fast, specific error message — never removed from the database, and never trusted to be enforced by the application alone.
2. **Entities mirror columns, not relationships.** No entity in this project has a `@ManyToOne` or `@OneToMany` JPA relationship; every foreign key is a plain typed field. Chapter 4 explains this trade-off in full.
3. **The database is the source of truth for structure.** Hibernate never generates or alters schema (`ddl-auto=none`); the SQL scripts in `db/` are authoritative, and entity-to-table correctness is verified by integration tests against a real PostgreSQL instance, not by Hibernate's own schema validator.

Chapter 4

The Persistence Layer

4.1 Entities Map Columns, Not Relationships

The single most consequential decision in this layer is one of omission: no entity in PharmaChain declares a `@ManyToOne`, `@OneToMany`, or `@ManyToMany` relationship. A `Batch` stores `productId` as a plain `String`, not a `ProductMaster` reference; a `Warehouse lot` stores `materialId` as a plain `String`, not a `MaterialMaster` reference. Every foreign key in the database is represented in Java as an ordinary typed field, and nothing more.

DESIGN DECISION

With nineteen tables and a genuinely interconnected schema (a batch touches products, materials, warehouse lots, QC reports, and dispensing records), a full JPA object graph would introduce lazy-loading configuration, N+1 query risk, and cascade-rule decisions across dozens of relationships — complexity that buys very little here, because the joins that actually matter for reporting already exist as SQL views (Section 2.2 and Chapter 2). `DashboardService` reads those views directly. Where a controller needs a related entity’s data (a product’s display name alongside a batch, for instance), it performs an explicit, visible lookup rather than relying on an implicit relationship traversal that hides a second query behind a getter call. This is a deliberate simplicity-over-object-graph trade-off, not an oversight, and it is one some teams make even in much larger systems, particularly for reporting-heavy or highly relational schemas where the object graph would be write-only overhead.

4.2 The PostgreSQL Identifier-Folding Trap

Section 2.2 flagged this briefly; it is worth a full, standalone treatment here because it is the single most likely mistake a newcomer would make mapping this exact schema to JPA, and because it fails silently at the DDL level and loudly (but only) at runtime.

The DDL declares tables and columns with unquoted, mixed-case identifiers:

```
CREATE TABLE Material_Master (  
    Material_ID    VARCHAR(20) PRIMARY KEY,  
    Material_Name  VARCHAR(30) NOT NULL,  
    ...  
);
```

PostgreSQL’s identifier-folding rule is simple but easy to forget: *any unquoted identifier is folded to lowercase at creation time*. The table above is not actually named `Material_Master` on disk — it is named `material_master`, with a column named `material_id`, not `Material_ID`. Every SQL client that queries case-insensitively (psql among them) hides this completely; typing `SELECT * FROM Material_Master` works identically to `SELECT * FROM material_master`, because the identifier you type is folded the same way before comparison.

Hibernate’s `@Column(name = ...)` does not fold anything — it uses the exact string given, quoted, as the column name to look for. Getting this wrong does not fail at build

time; it fails the first time the application actually issues a query, with an error to the effect of *column "Material_ID" does not exist*, easily confused for a typo rather than a case-folding mismatch.

```
@Entity
@Table(name = "material_master", schema = "pharma_manufacturing")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class MaterialMaster {

    @Id
    @Column(name = "material_id", length = 20)
    @NotBlank
    private String materialId;

    @Column(name = "material_name", length = 30, nullable = false)
    @NotBlank
    private String materialName;

    @Column(name = "shelf_life", nullable = false)
    @NotNull @Positive
    private Integer shelfLife;

    // isHazardous folds to "ishazardous" (no underscore - PostgreSQL just
    // lowercases the whole run of characters, camelCase and all)
    @Column(name = "ishazardous", nullable = false)
    private Boolean hazardous;

    // DB: DEFAULT 1000 CHECK (Reorder_Level > 0)
    @Column(name = "reorder_level")
    @Positive
    private BigDecimal reorderLevel;
}
```

Listing 4.1: Correct: explicit lowercase, snake_case column names matching what PostgreSQL actually stored

The `isHazardous` column is the sharpest edge of this rule: because it contains no underscore, PostgreSQL’s folding does not introduce one — the stored column name is `ishazardous`, a single run of lowercase characters, not `is_hazardous`. A developer guessing at the folded name by inserting underscores at every camelCase boundary (a reasonable but wrong mental model) would guess incorrectly for exactly this column.

WHY THIS MATTERS

This gotcha is exactly why `spring.jpa.hibernate.ddl-auto` is set to `none`, not `validate`, in `application.yml`. Hibernate’s own schema validator compares Java field types against actual column types using dialect-specific type-name compatibility rules that are difficult to predict without a live database connection to test against. Rather than fight that validator, entity-to-column correctness in this project is verified the way it should be verified — by an integration test that opens a real connection to a real, freshly-loaded PostgreSQL schema and actually reads and writes rows. Chapter 11 covers this in full.

4.3 Composite Keys via `@EmbeddedId`

Four tables — `RM_Transaction`, `Formula_Master`, `Material_Dispensing`, and `FG_Transaction` — are pure junction tables with a two-column composite primary key and no surrogate

identity column. JPA's standard tool for this is an `@Embeddable` class implementing the composite key, referenced from the owning entity via `@EmbeddedId`.

```
@Embeddable
@Getter @Setter @EqualsAndHashCode
@NoArgsConstructor @AllArgsConstructor @Builder
public class MaterialDispensingId implements Serializable {

    @Column(name = "batch_no")
    private Long batchNo;

    @Column(name = "item_id")
    private Long itemId;
}
```

Listing 4.2: The composite key for `Material_Dispensing`: (Batch_No, Item_ID)

```
@Entity
@Table(name = "material_dispensing", schema = "pharma_manufacturing")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class MaterialDispensing {

    @EmbeddedId
    private MaterialDispensingId id;

    @Column(name = "quantity_issued", nullable = false)
    private BigDecimal quantityIssued;
}
```

Listing 4.3: The owning entity, using the composite key

DESIGN DECISION

`@EqualsAndHashCode` on the embeddable key class is not optional decoration — JPA requires a composite identifier to implement value-based equality for entity identity comparison and for correct behavior inside Hibernate's persistence context (its first-level cache uses identity equality keyed by, among other things, this exact `equals()/hashCode()` pair). The owning entity itself deliberately does *not* override `equals()/hashCode()` — leaving JPA entities at default (reference) identity avoids a well-known family of subtle bugs around mutable fields and `Set` membership, and is the safer default absent a specific reason to do otherwise.

Spring Data JPA repositories over a composite key extend `JpaRepository<Entity, IdClass>` rather than `JpaRepository<Entity, Long>`, and derived query methods can reach *into* the embedded id by property-path, exactly as if it were a flat field:

```
public interface MaterialDispensingRepository
    extends JpaRepository<MaterialDispensing, MaterialDispensingId> {
    List<MaterialDispensing> findByIdBatchNo(Long batchNo);
}
```

4.4 The Repository Layer

Every entity gets exactly one repository interface, and almost none of them contain any method bodies — Spring Data JPA generates the implementation at runtime from the interface declaration alone.

```
public interface MaterialMasterRepository
    extends JpaRepository<MaterialMaster, String> {
}
```

Listing 4.4: The simplest possible repository - no custom methods needed

Where a controller needs more than `findAll`/`findById`, a derived query method is added by name alone — Spring Data parses the method signature and generates the corresponding query, no annotation or SQL string required:

```
public interface WarehouseRepository extends JpaRepository<Warehouse, Long> {
    List<Warehouse> findByMaterialId(String materialId);
    Optional<Warehouse> findByMaterialIdAndInvoiceNo(String materialId, Long
invoiceNo);
}
```

4.5 Entity Inventory

Table 4.1 lists every entity, its identifier strategy, and which of the persistence-layer patterns from this chapter it demonstrates.

BUG FOUND & FIXED DURING REVIEW

Every entity in the top half of Table 4.1 marked “Client-assigned” shares a single, serious latent defect that was only discovered during the formal architecture review documented in Chapter 12: Spring Data JPA’s `save()` silently *updates* an existing row instead of failing when the assigned id already exists, because it calls `merge()`, not `persist()`, whenever the entity’s `@Id` is non-null. Every one of these entities required an explicit `existsById` guard added to its owning service method as a result. The full mechanism, and why it is dangerous, is explained in Section 12.2.

Table 4.1: All 24 entity classes and their identifier strategy

Entity	Identifier	Notes
MaterialMaster	Client-assigned String	
AccountMaster	Client-assigned String	
EmployeeMaster	Client-assigned String	
EquipmentMaster	Client-assigned String	
ProductMaster	Client-assigned String	
Transaction	Client-assigned Long (invoiceNo)	Shared buy/sell ledger
SupplierContract	Client-assigned String	
Warehouse	GenerationType.IDENTIFY	Auto-generated itemId
RmTransaction	@EmbeddedId RmTransactionId	(invoiceNo, itemId)
MaterialQuality Check	Client-assigned String	
FormulaMaster	@EmbeddedId FormulaMasterId	(productId, materialId); no REST endpoint yet
Batch	Client-assigned Long	
Material Dispensing	@EmbeddedId Material DispensingId	(batchNo, itemId)
ProductionLog	GenerationType.IDENTIFY	Also carries @AssertTrue cross-field validation — see Chapter 12
ProductQuality Check	Client-assigned String	
FgTransaction	@EmbeddedId FgTransactionId	(invoiceNo, batchNo)
ProductRecall	Client-assigned String	Only ever written via the stored procedure
MaintenanceLog	Client-assigned String	
QcAuditLog	GenerationType.IDENTIFY (Integer)	SERIAL, not BIGINT - read-only from the app
AppUser	GenerationType.IDENTIFY	Security layer - Chapter 8

Chapter 5

DTOs and Validation Strategy

5.1 Two Kinds of Request Body

PharmaChain uses two different strategies for request/response bodies, chosen deliberately per endpoint rather than applied uniformly, and it is worth being explicit about which endpoints use which and why.

1. **Entities directly, for simple master-data CRUD.** `POST /api/v1/materials` accepts a `MaterialMaster` entity as its request body. For a resource whose API shape is genuinely one-to-one with its table — create, read, update, delete, nothing more — introducing a parallel DTO class would add a file and a mapping step that repeats the entity’s fields verbatim, for no behavioral benefit.
2. **Dedicated record DTOs, for multi-table workflows.** Creating a batch, dispensing material, submitting a QC result, recording a purchase or sale, and initiating a recall all touch more than one table, or accept a shape that is not simply “here is a full entity.” Each of these has its own request record in `dto.request`.

DESIGN DECISION

This is a pragmatic middle ground, not a purist one. The stricter, more conventional position — never expose a JPA entity directly over REST — has real merit (it decouples the API contract from the persistence model, so a column rename doesn’t automatically become a breaking API change). For this project’s scope, the added indirection was judged not worth it for the six resources whose API shape and table shape are identical; it *was* worth it everywhere the two shapes diverge, which is precisely where a DTO’s decoupling benefit is actually being used rather than paid for and left idle.

5.2 Records as Request DTOs

Every request DTO is a Java `record`, not a class with a builder or a mutable POJO. Records are immutable by construction, get `equals/hashCode/toString` for free, and — critically for this use case — their compact canonical constructor is exactly where Bean Validation annotations attach cleanly.

```
/**
 * stockQty is optional and defaults to batchSize (nothing sold yet at
 * creation time). Date rules (no future mfg date, >= 6 month shelf life)
 * are enforced by trg_strict_batch_dates regardless of what the service
 * layer checks.
 */
public record CreateBatchRequest (
    @NotNull Long batchNo,
    @NotNull @Positive BigDecimal batchSize,
    @NotNull LocalDate mfgDate,
    @NotNull LocalDate expDate,
    @NotBlank String productId,
    @PositiveOrZero BigDecimal stockQty
```

```

    } {
    }

```

Listing 5.1: A request DTO spanning validation, defaulting, and a code comment explaining what the database re-checks

```

/**
 * results should be PASSED or FAILED - trg_prevent_bad_sales reads this
 * value straight from the database before allowing any future
 * FG_Transaction sale of the batch.
 */
public record SubmitProductQcRequest (
    @NotBlank String reportId,
    @NotNull Long batchNo,
    @NotNull LocalDate analysisDate,
    @NotBlank String analystName,
    @NotNull @Positive BigDecimal sampleSize,
    @NotBlank String processState,
    @NotBlank String test,
    @NotBlank String limits,
    @NotBlank @Pattern(regex = "PASSED|FAILED",
        message = "must be PASSED or FAILED") String results,
    String empId
) {
}

```

Listing 5.2: A request DTO using `@Pattern` to mirror a domain rule the trigger itself depends on

Note the `@Pattern` constraint on `results` in Listing 5.2. It is not an arbitrary input restriction — it mirrors, at the boundary of the API, the exact set of values `trg_prevent_bad_sales` (Listing 2.2) treats as meaningful. A typo like "Passed" or "pass" would not match the trigger's exact-string comparison and would silently behave as "not passed," blocking every future sale of the batch with no obvious explanation; catching that at the API boundary with a clear 400 is far better than discovering it downstream as an inexplicably blocked sale weeks later.

5.3 Validation Gaps Are a Real Bug Class

Every `@Positive`, `@PositiveOrZero`, and `@Pattern` constraint in this project's DTOs and entities was not written once and left alone — Chapter 12 documents a systematic audit that cross-referenced *every* `CHECK` constraint in the database's DDL against the corresponding application-level validation, and found thirteen gaps where a field accepted a value at the API layer that the database would ultimately reject. Table 5.1 previews three representative examples; the full list, with the exact `CHECK` constraint each one closes, is in Section 12.3.1.

WHY THIS MATTERS

None of these gaps were capable of corrupting data — the database's own `CHECK` constraint was always the real backstop, exactly as the defense-in-depth philosophy from Chapter 3 intends. What each gap actually cost was the *quality* of the failure: without the application-level constraint, a bad value travelled all the way to the database, and came back as an opaque `DB_RULE_VIOLATION` after a wasted round-trip, instead of an immediate, specific 400 explaining exactly which field was wrong. Closing these gaps is a usability fix riding on top of a correctness guarantee that was never actually broken.

Table 5.1: A sample of validation gaps closed during the architecture review (full list in Chapter 12)

Field	Database constraint	Application-level fix
MaterialMaster.shelfLife	CHECK (Shelf_Life > 0)	Added @Positive
ProductMaster.salableOrSample	CHECK (... IN ('M','S'))	Added @Pattern(regexp = "M S")
CreateBatchRequest.stockQty	CHECK (Stock_Qty >= 0)	Added @PositiveOrZero

5.4 Cross-Field Validation

Bean Validation constraints on a single field cover most of this project's needs, but one rule — `Production_Log`'s CONSTRAINT `chk_time CHECK (End_Time > Start_Time)` — spans two fields. Bean Validation supports this via a class-level `@AssertTrue` method:

```
/**
 * Mirrors the DB's own CONSTRAINT chk_time CHECK (End_Time > Start_Time)
 * as a fast, friendly 400 instead of waiting for the round-trip to fail
 * as a 422. @JsonIgnore keeps this synthetic validation-only property out
 * of the JSON Jackson would otherwise generate for it.
 */
@AssertTrue(message = "endTime must be after startTime")
@JsonIgnore
public boolean isTimeRangeValid() {
    return startTime == null || endTime == null || endTime.isAfter(startTime);
}
```

Listing 5.3: Cross-field validation mirroring a multi-column CHECK constraint

The `@JsonIgnore` annotation is not incidental. Any method following the `isXxx()` naming convention looks, to Jackson, like the getter for a bean property named `xxx` — without `@JsonIgnore`, every GET response for a production log would serialize a spurious `"timeRangeValid": true` field that has no corresponding column and no meaning to a client.

5.5 Response DTOs

Response shapes follow the same pattern in reverse: the three dashboard views (Chapter 2) each get a dedicated response record, since their column sets are the product of a join and aggregation, not a single table:

```
public record ExpiryRiskRow(
    Long batchNo,
    String productName,
    LocalDate expDate,
    Integer daysRemaining,
    String riskStatus,
    BigDecimal manufacturedQty,
    BigDecimal totalSoldQty,
    BigDecimal unsoldInventory
) {
```

```
}
```

`DashboardService` populates these records directly from a `ResultSet` via an explicit `RowMapper` lambda (Chapter 6), rather than through `BeanPropertyRowMapper`'s reflection-based approach — records have no setters for reflection to find, and an explicit mapper is both faster to reason about and immune to a column being silently dropped from the mapping without a compile error.

Chapter 6

The Service Layer

6.1 Two Categories of Service

Consistent with the two categories of DTO strategy from Chapter 5, services in PharmaChain fall into two groups. Simple master-data services (`MaterialService`, `AccountService`, and five others) are thin wrappers with almost no logic of their own — their job is transaction demarcation and, as this chapter's closing section covers, a duplicate-id guard. Workflow services (`BatchService`, `QualityCheckService`, `PurchaseService`, `SalesService`, `RecallService`) contain the project's actual business logic, and every one of them follows the same shape: validate what can be validated quickly, then delegate to the database for the guarantee that actually matters.

6.2 A Complete Worked Example: BatchService

`BatchService` is the clearest illustration of the defense-in-depth pattern in the entire codebase, because it contains two workflows — creating a batch and dispensing material to it — each of which pairs an application-level check with a specific database trigger from Chapter 2.

```
/**
 * Creates a production batch. Mfg/Exp date rules are re-validated here
 * for a fast, friendly error, but trg_strict_batch_dates on the Batch
 * table is what actually guarantees they can never be violated - even
 * by a direct SQL client that bypasses this API entirely.
 */
@Transactional
public Batch createBatch(CreateBatchRequest request) {
    if (batchRepository.existsById(request.batchNo())) {
        throw new BusinessException(
            "Batch '" + request.batchNo() + "' already exists");
    }
    if (request.mfgDate().isAfter(LocalDate.now())) {
        throw new BusinessException(
            "Manufacturing date cannot be in the future: " + request.
mfgDate());
    }
    if (request.expDate().isBefore(request.mfgDate().plusMonths(6))) {
        throw new BusinessException(
            "Expiry date must be at least 6 months after the manufacturing
date");
    }

    Batch batch = Batch.builder()
        .batchNo(request.batchNo())
        .batchSize(request.batchSize())
        .mfgDate(request.mfgDate())
        .expDate(request.expDate())
        .productId(request.productId())
        .stockQty(request.stockQty() != null ? request.stockQty() : request.
batchSize())
        .utQA("UT")
```

```

        .build();
    return batchRepository.save(batch);
}

```

Listing 6.1: `BatchService.createBatch()` - full method

Three checks precede the actual write, and it is worth being precise about which ones are *redundant safety nets* versus which one is the *only* thing preventing a real bug:

- The manufacturing-date and expiry-date checks duplicate `trg_strict_batch_dates` exactly. If both were deleted, the trigger alone would still make this method perfectly safe — the only cost would be a less friendly error message on the failure path.
- The `existsById` check is not a duplicate of anything at the database level. Section 12.2 explains why: without it, a duplicate `batchNo` does not fail at all, it silently overwrites the existing batch. This check is load-bearing, not a nicety.

```

@Transactional
public MaterialDispensing dispenseMaterial(DispenseMaterialRequest request) {
    findById(request.batchNo());
    Warehouse lot = warehouseRepository.findById(request.itemId())
        .orElseThrow(() -> ResourceNotFoundException.forId("Warehouse item",
            request.itemId()));

    MaterialDispensingId dispensingId = MaterialDispensingId.builder()
        .batchNo(request.batchNo())
        .itemId(request.itemId())
        .build();
    if (dispensingRepository.existsById(dispensingId)) {
        throw new BusinessException(
            "Material has already been dispensed from item %d to batch %d - "
            + "dispense from a different lot instead"
            .formatted(request.itemId(), request.batchNo()));
    }

    if (lot.getStock().compareTo(request.quantityIssued()) < 0) {
        throw new BusinessException(
            "Not enough stock: item %d has %s units left, requested %s"
            .formatted(request.itemId(), lot.getStock(), request.
            quantityIssued()));
    }

    MaterialDispensing dispensing = MaterialDispensing.builder()
        .id(dispensingId)
        .quantityIssued(request.quantityIssued())
        .build();
    // trg_deduct_stock_on_dispense fires here and decrements Warehouse.stock
    // atomically.
    return dispensingRepository.save(dispensing);
}

```

Listing 6.2: `BatchService.dispenseMaterial()` - full method

Here the stock check (comparing `lot.getStock()` against the requested quantity) is the redundant one — `trg_deduct_stock_on_dispense` enforces exactly the same rule and would reject an over-dispense on its own. The composite-key existence check, once again, is not redundant: Section 12.2 explains the specific, more dangerous failure mode this particular check prevents, which involves the trigger not firing at all on the resulting operation.

6.3 JdbcTemplate for What JPA Doesn't Cover

Two responsibilities in this project deliberately bypass Spring Data JPA entirely, because JPA has no idiomatic first-class way to express them.

6.3.1 Reading the Views

```
public List<ExpiryRiskRow> expiryRisk() {
    return jdbcTemplate.query(
        "SELECT batch_no, product_name, exp_date, days_remaining,
        risk_status, "
        + "manufactured_qty, total_sold_qty, unsold_inventory "
        + "FROM pharma_manufacturing.v_inventory_expiry_risk",
        (rs, rowNum) -> new ExpiryRiskRow(
            rs.getLong("batch_no"),
            rs.getString("product_name"),
            rs.getObject("exp_date", LocalDate.class),
            rs.getInt("days_remaining"),
            rs.getString("risk_status"),
            rs.getBigDecimal("manufactured_qty"),
            rs.getBigDecimal("total_sold_qty"),
            rs.getBigDecimal("unsold_inventory")
        ));
}
```

Listing 6.3: DashboardService - direct SQL against a view, mapped by an explicit lambda

No business logic lives in this method — deliberately. The SQL view already performs the join and aggregation (Chapter 2); this method's only job is to run it and map each row onto a response record.

6.3.2 Calling the Stored Procedure

Spring Data JPA's `@Procedure` annotation supports PostgreSQL `FUNCTIONS`, not `PROCEDURES`, so `RecallService` calls `execute_product_recall` through a raw JDBC `CallableStatement` instead:

```
@Transactional
public void initiateRecall(InitiateRecallRequest request) {
    jdbcTemplate.execute((ConnectionCallback<Void>) connection -> {
        try (CallableStatement cs = connection.prepareCall(
            "{call execute_product_recall(?, ?, ?)}")) {
            cs.setString(1, request.recallId());
            cs.setLong(2, request.batchNo());
            cs.setString(3, request.reason());
            cs.execute();
        }
        return null;
    });
}
```

Listing 6.4: Calling a PostgreSQL `PROCEDURE` - the one thing Spring Data JPA cannot do directly

6.4 Transaction Boundaries

Every service in this project is annotated `@Transactional(readOnly = true)` at the class level, with individual write methods overriding to `@Transactional(read-write)`. This

means every read path gets Hibernate’s read-only optimizations by default, and every write path is explicit about the fact that it writes — a reviewer scanning method signatures can tell which operations mutate state without reading the method body.

For the multi-step workflows — `PurchaseService.recordPurchase()` writes to `Transactions`, `Warehouse`, and `RM_Transaction` in one call — the `@Transactional` boundary is what makes the whole operation atomic: if the third insert fails, the first two roll back with it, rather than leaving an orphaned invoice with no corresponding warehouse lot.

6.5 The Duplicate-ID Guard Pattern

Every `create`-style method in every service in this project begins with the same shape of check, introduced during the architecture review documented in Chapter 12:

```
if (repository.existsById(entity.getId())) {  
    throw new BusinessException("... already exists");  
}
```

This looks like defensive boilerplate. It is not — Section 12.2 explains, in full, the specific and non-obvious Spring Data JPA behavior that makes this check necessary everywhere a primary key is client-supplied rather than database-generated, and why omitting it does not produce an error at all, but a silent, incorrect success.

Chapter 7

The REST API and Exception Handling

7.1 Controller Conventions

Controllers in PharmaChain are kept deliberately thin: HTTP concerns only — routing, status codes, request validation via `@Valid`, and method-level authorization via `@PreAuthorize` (Chapter 8). No controller method contains business logic; every one delegates to exactly one service call.

```
@PostMapping("/{batchNo}/dispense")
@PreAuthorize("hasAnyRole('ADMIN','WAREHOUSE_MANAGER','PRODUCTION_SUPERVISOR')")

@ResponseStatus(HttpStatus.CREATED)
public MaterialDispensing dispense(@PathVariable Long batchNo,
                                   @Valid @RequestBody DispenseMaterialRequest
                                   request) {
    DispenseMaterialRequest scoped =
        new DispenseMaterialRequest(batchNo, request.itemId(), request.
        quantityIssued());
    return service.dispenseMaterial(scoped);
}
```

Listing 7.1: A representative workflow controller method

The `scoped` reconstruction on the line above is a small but deliberate detail: the batch number in the URL path is authoritative, and the request body’s own (redundant) `DispenseMaterialRequest.batchNo()` field, if a client sent a mismatched one, is discarded rather than trusted. A client cannot dispense material against a different batch than the one named in the URL by crafting a mismatched body.

7.2 The `ApiError` Contract

Every error response in this API, regardless of which layer produced it, has the identical shape:

```
public record ApiError(
    Instant timestamp,
    int status,
    String code,
    String message,
    List<String> details
) {
}
```

A consistent, machine-parseable error shape is what makes `GlobalExceptionHandler` — the subject of the rest of this chapter — worth building at all: a frontend or API consumer can branch on `code` without needing to distinguish “the database rejected this” from “a field failed validation” from “you don’t have permission,” beyond reading one field.

7.3 GlobalExceptionHandler: One Handler per Failure Mode

Every exception that can occur anywhere in the request lifecycle is mapped, by type, to a specific HTTP status and code. Table 7.1 is the complete map as it exists in the shipped source.

Table 7.1: Every exception type handled, and what it means

Exception	HTTP	Code	Meaning
ResourceNotFoundException	404	NOT_FOUND	Looked up an id that doesn't exist
BusinessRuleViolationException	422	BUSINESS_RULE_VIOLATION	Service-layer pre-check failed
MethodArgumentNotValidException	400	VALIDATION_FAILED	@Valid body failed Bean Validation
ConstraintViolationException	400	VALIDATION_FAILED	Bean Validation outside @Valid (defensive)
MethodArgumentTypeMismatchException	400	INVALID_PARAMETER	e.g. non-numeric path variable
HttpMessageNotReadableException	400	MALFORMED_REQUEST_BODY	Malformed JSON
NoHandlerFoundException	404	NOT_FOUND	No route matches at all
DataAccessException	422	DB_RULE_VIOLATION	A trigger or constraint rejected the write
AccessDeniedException	403	FORBIDDEN	Authenticated, wrong role
AuthenticationException	401	UNAUTHENTICATED	Bad credentials / no valid session
Exception (catch-all)	500	INTERNAL_ERROR	Genuinely unexpected

Three of these handlers are worth a closer look, because each one exists to correct a specific, non-obvious failure that would otherwise have surfaced as an uninformative 500.

7.3.1 Unwrapping a Trigger's Message from DataAccessException

```
/**
 * Catches everything that comes back from the database layer -
 * unique/foreign-key/CHECK constraint violations, and the custom
 * RAISE EXCEPTION messages thrown by the four PL/pgSQL triggers.
 * Postgres reports RAISE EXCEPTION under SQLSTATE P0001, which Spring's
 * default translator doesn't map to a specific DataAccessException
 * subtype, so this handler targets the broad DataAccessException and
 * unwraps the root PSQLErrorException itself to recover the human-readable
 * message the trigger actually raised.
 */
@ExceptionHandler(DataAccessException.class)
public ResponseEntity<ApiError> handleDataAccess(DataAccessException ex) {
    Throwable root = rootCause(ex);
    String message = (root instanceof PSQLErrorException psql
        && psql.getServerErrorMessage() != null)
        ? psql.getServerErrorMessage().getMessage()
        : "The request violates a database business rule";
    return ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY)
        .body(ApiError.of(422, "DB_RULE_VIOLATION", message));
}
```

Listing 7.2: Recovering a trigger's RAISE EXCEPTION message from a wrapped SQLException

This is the single handler that makes every trigger from Chapter 2 visible to an API client as a normal, readable 422 rather than a stack trace. Without it, a well-behaved attempt to over-dispense material would still be correctly *blocked* (the trigger still fires), but the client would receive an opaque 500 with no indication of why.

7.3.2 The @PreAuthorize Ordering Surprise

```
/**
 * Handles @PreAuthorize denials. Because method security is an AOP proxy
 * around the controller/service bean, this exception is thrown during
 * handler invocation - inside DispatcherServlet's own try/catch - so it
 * reaches this @RestControllerAdvice before it would ever reach the
 * security filter chain's ExceptionTranslationFilter. Without this
 * handler, an authenticated user with the wrong role would see a
 * generic 500, not a 403.
 */
@ExceptionHandler(org.springframework.security.access.AccessDeniedException.
    class)
public ResponseEntity<ApiError> handleAccessDenied(
    org.springframework.security.access.AccessDeniedException ex) {
    return ResponseEntity.status(HttpStatus.FORBIDDEN)
        .body(ApiError.of(403, "FORBIDDEN",
            "Your account role does not have permission to perform this
            action"));
}
```

Listing 7.3: A handler that exists because of a genuinely counter-intuitive Spring MVC/Security interaction

Chapter 8 explains this ordering interaction in full: Spring Security ships its own Access DeniedHandler mechanism (RestAccessDeniedHandler in this project), but it only intercepts exceptions that escape the entire filter chain. An @PreAuthorize check on a controller method is an AOP proxy wrapping that method, and the resulting AccessDeniedException is thrown *during* Spring MVC's own handler invocation — which means @RestControllerAdvice sees it first. Both paths (a filter-chain-level denial for an entirely unauthenticated request, and a method-level denial for an authenticated-but-wrong-role request) needed their own handler for the API to be consistent regardless of *which* mechanism caught the problem.

7.3.3 Client-Input Edge Cases

```
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<ApiError> handleTypeMismatch(
    MethodArgumentTypeMismatchException ex) {
    String expected = ex.getRequiredType() != null
        ? ex.getRequiredType().getSimpleName() : "a different type";
    return ResponseEntity.status(HttpStatus.BAD_REQUEST)
        .body(ApiError.of(400, "INVALID_PARAMETER",
            "'%s' is not a valid value for '%s' - expected %s"
            .formatted(ex.getValue(), ex.getName(), expected)));
}
```

Listing 7.4: Turning a type-conversion failure into a specific 400

A request like `GET /api/v1/batches/not-a-number`, where `{batchNo}` is declared as a `Long`, fails during Spring MVC's own argument-resolution step, before the controller method body ever runs. Without a dedicated handler, this exception — along with malformed JSON bodies, handled identically via `HttpMessageNotReadableException` — fell through to the generic `ExceptionHandler` handler and returned a misleading 500 for what is, unambiguously, a client mistake rather than a server fault.

WHY THIS MATTERS

Every handler in Table 7.1 targets a distinct, non-overlapping exception type in the class hierarchy, which matters for a subtle reason: Spring resolves `ExceptionHandler` methods using `ExceptionDepthComparator`, picking whichever declared handler is *closest* in the exception's inheritance chain, regardless of the order the handler methods appear in the source file. Two handlers registered for types with an ambiguous common distance from the thrown exception would make Spring Boot fail at startup with `IllegalStateException: Ambiguous @ExceptionHandler method mapped`. Every type in this table was chosen, and cross-checked, specifically to avoid that.

Chapter 8

Authentication and Authorization

8.1 Design Goals

The security layer needed to satisfy three requirements that shaped every decision in this chapter: it had to be stateless (no server-side session storage, so the API scales horizontally without sticky sessions), it had to express role-based access control against a small, fixed set of operational roles rather than free-text HR data, and it had to leave the original database schema untouched. JSON Web Tokens satisfy the first requirement directly; the second and third shaped the schema decision in Section 8.2.

8.2 A New Table, Deliberately Separate from `Employee_Master`

It would have been possible to add `username` and `password_hash` columns directly to `Employee_Master`. This was rejected: `Employee_Master` is HR data — who works here, which department, when they were hired — and login credentials are a different concern with a different lifecycle (a login can be disabled without touching the employee record; a service account might need a login with no corresponding employee at all). A new table, `app_user`, was added instead, layered on top of the original schema without modifying it:

```
CREATE TABLE app_user (  
  user_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  username     VARCHAR(50) UNIQUE NOT NULL,  
  password_hash VARCHAR(100) NOT NULL,  
  emp_id       VARCHAR(20) REFERENCES employee_master(emp_id),  
  role         VARCHAR(30) NOT NULL  
              CHECK (role IN ('ADMIN', 'QC_ANALYST', 'WAREHOUSE_MANAGER',  
                              'PRODUCTION_SUPERVISOR', 'SALES', 'AUDITOR')),  
  
  enabled      BOOLEAN NOT NULL DEFAULT TRUE,  
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
);
```

Listing 8.1: `app_user`: a new table, loosely linked to the original schema

`emp_id` is a nullable foreign key, not a mandatory one — a login account can optionally correspond to a real employee, but is not required to. The `role` CHECK constraint fixes the set of valid roles at the database level, deliberately independent of whatever free-text values happen to live in `Employee_Master.Role`, which was never designed as an authorization scheme.

8.3 Issuing and Validating Tokens

```
@Component  
public class JwtService {  
  
  private final SecretKey key;  
  private final long expirationMs;
```

```

public JwtService(@Value("${app.jwt.secret}") String secret,
                  @Value("${app.jwt.expiration-ms}") long expirationMs) {
    this.key = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
    this.expirationMs = expirationMs;
}

public String generateToken(String username, String role) {
    Date now = new Date();
    Date expiry = new Date(now.getTime() + expirationMs);
    return Jwts.builder()
        .subject(username)
        .claim("role", role)
        .issuedAt(now)
        .expiration(expiry)
        .signWith(key)
        .compact();
}
// extractUsername(), extractRole() omitted - both parse and verify
// the signature, returning Optional.empty() on any failure.
}

```

Listing 8.2: JwtService - HS256 token issuance via jjwt 0.13.0

DESIGN DECISION

The token's role claim is informational, not authoritative. `JwtAuthenticationFilter` (Section 8.4) never reads it directly for authorization decisions — every request re-loads the user's *current* role from the database via `AppUserDetailsService`. This means an administrator changing a user's role takes effect on that user's very next request, not after their existing token happens to expire (which could be up to an hour later, per the default `JWT_EXPIRATION_MS`).

8.4 The Authentication Filter

```

@Component
@RequiredArgsConstructor
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response, FilterChain filterChain)
        throws ServletException, IOException {

        String header = request.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer "))
            && SecurityContextHolder.getContext().getAuthentication() ==
        null) {

            String token = header.substring(7);
            Optional<String> username = jwtService.extractUsername(token);

            if (username.isPresent()) {
                try {

```

```

        UserDetails userDetails = userDetailsService.
loadUserByUsername(username.get());
        if (isUsable(userDetails)) {
            var authToken = new UsernamePasswordAuthenticationToken
(
                userDetails, null, userDetails.getAuthorities())
;
            authToken.setDetails(new WebAuthenticationDetailsSource
().buildDetails(request));
            SecurityContextHolder.getContext().setAuthentication(
authToken);
        }
        // else: disabled/locked/expired - fall through to a normal
401.
    } catch (UsernameNotFoundException e) {
        // token names a deleted user - fall through to a normal
401.
    }
}
filterChain.doFilter(request, response);
}

private boolean isUsable(UserDetails userDetails) {
    return userDetails.isEnabled() && userDetails.isAccountNonLocked()
        && userDetails.isAccountNonExpired() && userDetails.
isCredentialsNonExpired();
}
}

```

Listing 8.3: JwtAuthenticationFilter - the version shipped after the architecture review

This is not the version that was originally written. Two defects in the original implementation were found during the architecture review documented fully in Chapter 12, and are worth summarizing here because they explain exactly why the `isUsable` check and the `try/catch` exist.

BUG: A disabled or deleted account's token kept working

The original filter loaded `UserDetails` and authenticated the request without ever checking `isEnabled()`. Disabling a user's `app_user.enabled` flag had no effect on tokens already issued to them — they remained valid, and were accepted, until they expired on their own schedule. Separately, if the token named a user who had since been *deleted*, `loadUserByUsername` throws `UsernameNotFoundException` — and because this filter runs *before* `ExceptionTranslationFilter` in Spring Security's filter chain, nothing downstream would have caught it. It would have escaped as an unhandled exception producing the container's default error page, not this project's `ApiError` JSON shape. Both problems are closed by the same two changes: the `isUsable` guard, and the local `try/catch(UsernameNotFoundException)` that lets the request fall through to a normal 401 instead of an unhandled failure or a silent bypass.

8.5 Loading Users

```

@Service
@RequiredArgsConstructor

```



```

public class AppUserDetailsService implements UserDetailsService {

    private final AppUserRepository repository;

    @Override
    public UserDetails loadUserByUsername(String username) throws
    UsernameNotFoundException {
        AppUser user = repository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("No such user:
            " + username));

        return User.withUsername(user.getUsername())
            .password(user.getPasswordHash())
            .authorities("ROLE_" + user.getRole())
            .disabled(!Boolean.TRUE.equals(user.getEnabled()))
            .build();
    }
}

```

Listing 8.4: AppUserDetailsService - maps a database row to Spring Security's UserDetails

The single authority granted is "ROLE_" + role — Spring Security's `hasRole('X')` expression matches against an authority literally named `ROLE_X`, so this one-to-one mapping from the database's role column to a Spring Security authority is all that method-level security (Section 8.7) needs.

8.6 The Filter Chain Configuration

```

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
@RequiredArgsConstructor
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    private final RestAuthenticationEntryPoint authenticationEntryPoint;
    private final RestAccessDeniedHandler accessDeniedHandler;
    private final AppUserDetailsService userDetailsService;

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
    Exception {
        http
            .cors(cors -> cors.configurationSource(corsConfigurationSource()))
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s.sessionCreationPolicy(
            SessionCreationPolicy.STATELESS))
            .exceptionHandling(h -> h
                .authenticationEntryPoint(authenticationEntryPoint)
                .accessDeniedHandler(accessDeniedHandler))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/login").permitAll()
                .requestMatchers("/swagger-ui/**", "/swagger-ui.html",
                    "/api-docs/**", "/v3/api-docs/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .anyRequest().authenticated())
            .addFilterBefore(jwtAuthenticationFilter,
            UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}

```

```

    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setUserDetailsService(userDetailsService);
        provider.setPasswordEncoder(new BCryptPasswordEncoder());
        return provider;
    }
    // passwordEncoder(), authenticationManager(), corsConfigurationSource()
    omitted
}

```

Listing 8.5: SecurityConfig - stateless sessions, method security enabled, CORS

`/api/v1/auth/login`, the Swagger/OpenAPI paths, and `/actuator/health` are the only routes reachable without a token — the last of these specifically because Docker’s HEALTHCHECK and docker-compose’s `service_healthy` condition (Chapter 10) have no way to supply one. Everything else requires authentication as the coarse gate; `@EnableMethodSecurity` activates the finer-grained, per-endpoint `@PreAuthorize` checks covered in Section 8.7.

COURSE CONCEPT APPLIED

Explicitly defining a `DaoAuthenticationProvider` bean here means Spring Boot’s autoconfiguration uses exactly that one provider, rather than *also* constructing a redundant implicit default from the `UserDetailsService` and `PasswordEncoder` beans that also exist in the context. Spring Security’s bean-based `AuthenticationManagerBuilder` steps back and defers to an explicitly-provided `AuthenticationProvider` bean rather than building a second, competing one.

8.6.1 A Genuine CORS Subtlety

```

configuration.setAllowedOriginPatterns(List.of("*")); // not
               setAllowedOrigins
configuration.setAllowCredentials(true);

```

Combining a wildcard origin with `allowCredentials(true)` is, per the CORS specification, only valid if the server reflects the actual requesting origin in its response rather than a literal `"*"` — browsers reject the literal-wildcard-plus-credentials combination outright. `setAllowedOriginPatterns` (as opposed to `setAllowedOrigins`) is Spring’s API specifically for this case: internally, it reflects the real request origin back, satisfying the browser’s requirement while still accepting any origin. Using the wrong one of these two nearly identically-named methods would silently break every credentialed cross-origin request; this was verified correct during the architecture review rather than assumed.

8.7 Role-Based Access Control on Every Write Endpoint

Table 8.1 is the authoritative map of which role can perform which write operation, as implemented via `@PreAuthorize` across every controller.

Table 8.1: Role-based write access, by resource

Resource / action	Allowed roles
Materials, Supplier Contracts (create/update)	ADMIN, WAREHOUSE_MANAGER
Accounts, Employees (create/update/delete)	ADMIN only
Equipment (create/update)	ADMIN, WAREHOUSE_MANAGER, PRODUCTION_SUPERVISOR
Products (create/update)	ADMIN, PRODUCTION_SUPERVISOR
Any master-data delete	ADMIN only
Batch creation, material dispensing	ADMIN, WAREHOUSE_MANAGER, PRODUCTION_SUPERVISOR
Material & product QC submission	ADMIN, QC_ANALYST
Recording a purchase	ADMIN, WAREHOUSE_MANAGER
Recording a sale	ADMIN, SALES
Initiating a recall	ADMIN, QC_ANALYST
Maintenance logs, production logs	ADMIN, PRODUCTION_SUPERVISOR (+WAREHOUSE_MANAGER for maintenance)
Account registration	ADMIN only
Every GET endpoint	any authenticated role, including AUDITOR

BUG: Master-data writes had no role restriction at all

The middle rows of Table 8.1 did not always look like this. In the original implementation, `MaterialController`, `AccountController`, `EmployeeController`, `EquipmentController`, `ProductController`, and `SupplierContractController`'s create/update methods carried no `@PreAuthorize` at all — only the filter chain's base “must be authenticated” rule applied. Concretely, the `AUDITOR` role, whose entire purpose is read-only oversight, could create or edit Employee records, the raw-material catalog, the finished-goods catalog, supplier contracts, and financial account records. So could `SALES`, or any other low-privilege role. There was not even a demo account seeded for `AUDITOR` to exercise the restriction against, which was itself part of the fix: a fifth demo account (`auditor / Audit@123`) was added specifically so this boundary is exercised by an integration test (Chapter 11), not merely assumed correct.

8.8 Login

```
public AuthResponse login(LoginRequest request) {
    try {
        authenticationManager.authenticate(
            new UsernamePasswordAuthenticationToken(request.username(),
            request.password()));
    } catch (AuthenticationException e) {
        throw new BadCredentialsException("Invalid username or password");
    }

    AppUser user = userRepository.findByUsername(request.username())
        .orElseThrow(() -> new BadCredentialsException("Invalid username or
        password"));
}
```

```
String token = jwtService.generateToken(user.getUsername(), user.getRole());  
  
return AuthResponse.bearer(token, user.getUsername(), user.getRole(),  
    jwtService.expirationSeconds());  
}
```

Listing 8.6: `AuthService.login()` - delegates credential checking to the framework

WHY THIS MATTERS

Every authentication failure — wrong password, unknown username, or a disabled account — produces the identical message: *Invalid username or password*. This is intentional, not an oversight discovered during review. Returning a more specific message per failure type (“that account is disabled,” for instance) would let an attacker enumerate valid usernames or probe account status without ever guessing a correct password. The generic message is the safer default for a login endpoint, even at some cost to a legitimate disabled user’s ability to self-diagnose why their login failed.

8.9 Demo Accounts

Five accounts are seeded for local development and grading, one per role, listed in full in Appendix A. Passwords are stored as genuine BCrypt hashes (strength 10) — generated once and committed as literal hash strings in `db/02_security_schema.sql` — rather than hashed by application code on first run, so the seed data works identically whether the schema is loaded via `psql` directly or through the Docker Compose stack’s automatic `docker-entrypoint-initdb.d` mechanism (Chapter 10).

Chapter 9

The Spring AI Compliance Copilot

9.1 What Was Built

Two AI-powered capabilities sit alongside the REST API, both built on Spring AI 1.0.1: a compliance copilot that answers natural-language questions by combining retrieval-augmented generation with live tool-calling, and a structured-output recall-notice generator that drafts a formal document from a single real database record.

9.2 A Deliberate Provider Split

Chat and embeddings come from two different providers, which is unusual enough to justify explaining directly.

DESIGN DECISION

Anthropic's API has no embeddings endpoint at all, so a chat-only Anthropic integration cannot power retrieval-augmented generation on its own. Rather than route embeddings through a second paid provider, PharmaChain uses a local Ollama model (`nomic-embed-text`, 768 dimensions) for embeddings exclusively, while chat continues to use Anthropic. This keeps the RAG pipeline runnable with zero additional API cost beyond whatever the chat calls themselves cost, at the price of requiring Ollama installed locally for the AI features specifically — a trade-off documented explicitly in the project's `README.md` rather than left for a developer to discover the hard way.

Having two `ChatModel`-family beans in the same Spring context (Anthropic's and Ollama's) creates a real ambiguity risk for anything that asks Spring for a generic `ChatModel` or relies on Spring AI's own autoconfigured `ChatClient.Builder`. `AiConfig` sidesteps this by asking for the concrete `AnthropicChatModel` type explicitly:

```
/**
 * Both spring-ai-starter-model-anthropic and spring-ai-starter-model-ollama
 * are on the classpath (Anthropic for chat, Ollama purely for local/free
 * embeddings), which means the context ends up with two ChatModel-family
 * beans. Rather than lean on Spring AI's own auto-configured
 * ChatClient.Builder (which would be ambiguous about which ChatModel to
 * use), this bean is wired explicitly to the concrete AnthropicChatModel
 * type, sidestepping the ambiguity entirely.
 */
@Configuration
public class AiConfig {

    private static final String SYSTEM_PROMPT = """
        You are the PharmaChain compliance copilot, embedded in a
        pharmaceutical manufacturing system. You answer questions about
        inventory, batches, quality control, and recalls using the
        tools and reference documents you're given.

        Rules:
        - Ground every factual claim in the tool results or retrieved
```

```

        documents you were given for this request. If you don't have
        the information, say so plainly instead of guessing.
        - When asked why something is blocked or not allowed, explain
        the specific business rule responsible, in plain English.
        - Be concise. This is an operational tool, not a chat companion.
        """;

@Bean
public ChatClient chatClient(AnthropicChatModel anthropicChatModel) {
    return ChatClient.builder(anthropicChatModel)
        .defaultSystem(SYSTEM_PROMPT)
        .build();
}
}

```

Listing 9.1: AiConfig - resolving a genuine bean-ambiguity risk by asking for a concrete type

The system prompt's first rule — ground every claim in retrieved content or tool results, and say so plainly rather than guess — is not boilerplate. It is the single instruction doing the most work to keep the compliance copilot from confidently fabricating an answer about, say, a batch's QC status when the tool call to check it was never made.

9.3 Tool-Calling Instead of Raw SQL Access

The most consequential AI-safety decision in this chapter is what the model is *not* given: direct SQL execution capability. Instead, the same three dashboard views from Chapter 2 are exposed as @Tool-annotated methods.

```

/**
 * These methods are what the compliance copilot calls to answer questions
 * about current data, instead of being handed raw SQL access. Each one
 * just delegates to DashboardService - the same read layer the REST
 * dashboard endpoints use - so the model can never see or touch anything
 * the API itself couldn't already return.
 */
@Component
@RequiredArgsConstructor
public class DashboardAiTools {

    private final DashboardService dashboardService;

    @Tool(description = "Get every raw material whose warehouse stock has "
        + "dropped to or below its configured reorder level")
    public List<InventoryShortageRow> getInventoryShortage() {
        return dashboardService.inventoryShortage();
    }

    @Tool(description = "Get every unsold finished-goods batch along with "
        + "how many days remain until it expires")
    public List<ExpiryRiskRow> getExpiryRisk() {
        return dashboardService.expiryRisk();
    }

    @Tool(description = "Get the full traceability record for one "
        + "production batch: product, dates, QC status, materials, sales")
    public List<TraceabilityRow> getBatchTraceability(
        @ToolParam(description = "The batch number to trace, e.g. 5001")
        Long batchNo) {
        return dashboardService.traceability(batchNo);
    }
}

```

}

Listing 9.2: DashboardAiTools - the model's only window into live data**WHY THIS MATTERS**

Giving a language model unrestricted SQL execution ability is a well-known class of risk — not only conventional injection concerns, but the model itself choosing to run an expensive, overly-broad, or destructive query in pursuit of answering a question reasonably. Constraining it to three named, single-purpose, read-only methods means the compliance copilot can only ever return data the REST API could already return through `DashboardService` directly. There is no query the model can construct that reaches further than what these three tools expose.

9.4 Retrieval-Augmented Generation

9.4.1 The Knowledge Base

Seven Markdown documents live in `src/main/resources/compliance-docs/`, each one explaining — in plain English, written specifically for this purpose — why one of the database objects from Chapter 2 exists and what it does: one per trigger, one covering all three views together, one for the recall procedure, and one connecting the whole design to FDA 21 CFR Part 11 at a conceptual level. These are not auto-generated from code comments; they are written as a genuine knowledge base, because a retrieval system is only as good as what it has to retrieve.

9.4.2 Ingestion

```
@Component
@RequiredArgsConstructor
@Slf4j
@ConditionalOnProperty(
    name = "app.ai.ingest-compliance-docs-on-startup",
    havingValue = "true", matchIfMissing = true)
public class ComplianceDocsIngestionRunner implements ApplicationRunner {

    private final VectorStore vectorStore;
    private final JdbcTemplate jdbcTemplate;

    @Value("classpath:compliance-docs/*.md")
    private Resource[] complianceDocs;

    @Override
    public void run(ApplicationArguments args) {
        try {
            Integer existing = jdbcTemplate.queryForObject(
                "SELECT COUNT(*) FROM public.compliance_docs", Integer.
class);
            if (existing != null && existing > 0) {
                log.info("Compliance docs already ingested ({} rows) - skipping", existing);
                return;
            }
            List<Document> documents = new ArrayList<>();
            for (Resource resource : complianceDocs) {
```



```

        String content = resource.getContentAsString(StandardCharsets.
UTF_8);
        documents.add(new Document(content,
            Map.of("source", String.valueOf(resource.getFilename()))
));
    }
    vectorStore.add(documents);
    log.info("Ingested {} compliance documents", documents.size());
} catch (Exception e) {
    log.warn("Skipping compliance-doc ingestion - is Ollama running? "
        + "The rest of the API is unaffected. Cause: {}", e.
getMessage());
}
}
}

```

Listing 9.3: ComplianceDocsIngestionRunner - defensive by design

Three defensive properties are worth calling out explicitly, because each one closes a specific failure mode:

- **Idempotent.** A row-count check means restarting the application does not re-ingest (and duplicate) the same seven documents.
- **Individually disableable.** `@ConditionalOnProperty` lets the integration test suite (Chapter 11) turn this runner off entirely via a single property, since the test environment has no Ollama instance to embed against.
- **Failure-tolerant.** The `try/catch` around the entire body means an unreachable Ollama instance produces a logged warning, not a failed application startup. The AI features degrade gracefully; nothing else in the API depends on them.

9.4.3 A Schema-Placement Decision That Prevented a Startup-Ordering Bug

The vector store’s table is configured to live in PostgreSQL’s default `public` schema, not in `pharma_manufacturing` alongside the rest of the project’s tables. This was not the original plan.

CAUGHT DURING DESIGN: A startup-ordering hazard

PgVectorStore’s schema initialization (`CREATE EXTENSION IF NOT EXISTS vector, CREATE TABLE` for its own storage) runs during Spring context startup, as part of constructing the `VectorStore` bean. If that table were configured to live in `pharma_manufacturing`, its creation would depend on that schema already existing — but in the Testcontainers integration test (Chapter 11), the business schema is loaded *after* the Spring context has already started, via a `@BeforeAll` step, precisely the wrong order. Even in ordinary local development, a developer who ran `mvn spring-boot:run` before loading `db/01_schema_and_data.sql` would have hit the same failure. Keeping the vector store in the default `public` schema, which always exists in a fresh PostgreSQL database, removes the ordering dependency entirely — and is arguably the more correct modeling choice regardless, since the vector store is a Spring-AI-managed concern, not part of the hand-designed business schema.

A closely related fix: the standard `postgres:16-alpine` image used for local development and integration testing does not ship the `pgvector` extension. Both `docker-compose.yml`

and the Testcontainers configuration were switched to the `pgvector/pgvector:pg16` image — a drop-in PostgreSQL 16 build with the extension pre-installed — which would otherwise have made every `CREATE EXTENSION vector` call fail outright.

9.4.4 Combining Retrieval and Tools in One Call

```
/**
 * A single chat call does two things at once: QuestionAnswerAdvisor
 * retrieves the most relevant compliance documents from pgvector and
 * stuffs them into the prompt, while DashboardAiTools lets the model pull
 * live numbers if the question needs them. A question like "why can't I
 * sell batch 5003" ends up grounded in both: the retrieved
 * trg_prevent_bad_sales explanation *and* a live lookup of batch 5003's
 * actual QC status.
 */
@Service
@RequiredArgsConstructor
public class ComplianceCopilotService {

    private final ChatClient chatClient;
    private final VectorStore vectorStore;
    private final DashboardAiTools dashboardAiTools;

    public String ask(String question) {
        QuestionAnswerAdvisor ragAdvisor = QuestionAnswerAdvisor.builder(
            vectorStore)
            .searchRequest(SearchRequest.builder()
                .topK(4)
                .similarityThreshold(0.5)
                .build())
            .build();

        return chatClient.prompt()
            .user(question)
            .advisors(ragAdvisor)
            .tools(dashboardAiTools)
            .call()
            .content();
    }
}
```

Listing 9.4: ComplianceCopilotService - RAG and tool-calling in a single chat call

`QuestionAnswerAdvisor` is a Spring AI *advisor*: middleware that intercepts a chat request, in this case to run a similarity search against `vectorStore` and inject the top four matching document chunks (`topK(4)`, above a `0.5` similarity threshold) directly into the prompt before the model ever sees it. Retrieval and tool-calling are independent mechanisms composed in the same request — the advisor supplies static, pre-written explanatory content; the tools supply live, queryable data. A well-formed question can draw on both in the same answer.

9.5 Structured Output: The Recall Notice Generator

The second AI feature demonstrates a different Spring AI capability: coercing a model's response into a specific, strongly-typed Java shape.

```
/**
```

```

* The shape ChatClient.call().entity(RecallNoticeDraft.class) asks the
* model to fill in. Spring AI generates a JSON schema from this record
* and instructs the model to respond in that exact shape, then parses
* the response back into a real object.
*/
public record RecallNoticeDraft(
    String title,
    String summary,
    String affectedProduct,
    String batchInformation,
    String consumerInstructions,
    String regulatoryNote
) {
}

```

Listing 9.5: The target shape - a plain record, no JSON-parsing boilerplate anywhere

```

public RecallNoticeDraft draftNotice(String recallId) {
    ProductRecall recall = recallRepository.findById(recallId)
        .orElseThrow(() -> ResourceNotFoundException.forId("Recall",
recallId));
    Batch batch = batchRepository.findById(recall.getBatchNo())
        .orElseThrow(() -> ResourceNotFoundException.forId("Batch", recall.
getBatchNo()));

    String productName = /* looked up from ProductMaster via batch.getProductId
() */;

    String prompt = """
        Draft a formal, regulator-ready product recall notice using
        ONLY the facts below. Do not invent any fact that isn't given
        here - if something isn't provided, write it generically
        rather than making it up.

        Recall ID: %s
        Reason for recall: %s
        Batch number: %d
        Product: %s
        Manufacturing date: %s
        Quantity recalled: %s
        Date initiated: %s
        """.formatted(recall.getRecallId(), recall.getReason(), batch.
getBatchNo(),
                    productName, batch.getMfgDate(), recall.getQtyRecalled(),
                    recall.getDateInitiated());

    return chatClient.prompt()
        .user(prompt)
        .call()
        .entity(RecallNoticeDraft.class);
}

```

Listing 9.6: RecallNoticeService - the prompt is built entirely from real data, never freeform caller input

DESIGN DECISION

Every fact in the prompt is read from the database via the recall's id — the caller supplies only `recallId`, never freeform text that ends up in the prompt. Combined with the explicit “do not invent any fact that isn't given here” instruction, this keeps the

model's role strictly to drafting and tone, not fact generation, for a document category (a regulatory recall notice) where a fabricated detail would be a genuinely serious problem, not merely an awkward one.

9.6 Endpoint Summary

Table 9.1: The two AI endpoints

Endpoint	Access	What it does
POST /api/v1/ai/ask	any authenticated role	RAG + tool-calling compliance Q&A
GET /api/v1/ai/recall-notice/{recallId}	ADMIN, QC_ANALYST	Structured-output recall notice draft

Chapter 10

Containerization and CI/CD

10.1 A Multi-Stage Dockerfile

The Dockerfile builds in two stages: a Maven/JDK image that compiles and packages the application, and a minimal JRE-only image that runs it. Nothing from the build stage — Maven itself, the dependency cache, source files — survives into the runtime image; only the packaged jar is copied across.

```
FROM maven:3.9-eclipse-temurin-21-noble AS build
WORKDIR /build

# Copy the POM first and resolve dependencies before copying source, so
# that a source-only change doesn't bust the dependency-download layer
# cache (this is the single biggest lever for fast repeat builds with
# Maven in Docker).
COPY pom.xml .
RUN mvn -B -q dependency:go-offline

COPY src ./src
RUN mvn -B -q clean package -DskipTests && rm -f target/*.jar.original
```

Listing 10.1: The build stage - dependency layer caching

DESIGN DECISION

Tests are deliberately *not* run as part of the image build. `BusinessRuleIntegrationTest` needs a real Docker daemon (Testcontainers boots an actual PostgreSQL container), and a Docker build sandbox does not provide one without extra plumbing — Docker-in-Docker, or mounting the host's Docker socket into the build container — that introduces real security trade-offs for comparatively little benefit. Tests run as their own dedicated CI job instead, directly on the runner against the runner's native Docker daemon (Section 10.4), which is both simpler and more secure than trying to run Testcontainers *during* an image build.

```
FROM eclipse-temurin:21-jre-noble
WORKDIR /app

RUN groupadd --system spring && useradd --system --gid spring --no-create-home
    spring

COPY --from=build /build/target/pharmachain-backend-*.jar app.jar
RUN chown spring:spring app.jar
USER spring:spring

EXPOSE 8080

# Real HTTP health check against Actuator's /actuator/health, sent by hand
# over bash's /dev/tcp pseudo-device rather than via curl/wget - the
# noble-based JRE image doesn't ship either by default.
HEALTHCHECK --interval=30s --timeout=5s --start-period=45s --retries=3 \
```

```

CMD bash -c '\
  exec 3<>/dev/tcp/127.0.0.1/8080 && \
  printf "GET /actuator/health HTTP/1.1\r\nHost: localhost\r\nConnection:
close\r\n\r\n" >&3 && \
  grep -q "\"status\": \"UP\"" <&3' || exit 1

ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]

```

Listing 10.2: The runtime stage - minimal image, non-root user, real health check

Three details in the runtime stage are each closing a specific, real risk rather than following a checklist:

- **A dedicated, unprivileged `spring` user.** The container never runs as `root` — a compromised application process inside the container has no more privilege than an ordinary unprivileged Linux user.
- **A real HTTP health check, not a bare TCP probe.** An earlier version of this `HEALTHCHECK` only verified that *something* was listening on port 8080, which would report healthy even if the application had started but was failing every request. Once `spring-boot-starter-actuator` was added (Section 10.2), the check was upgraded to actually parse the JSON health response body for a literal `"status": "UP"`.
- **Shell-form `ENTRYPOINT`.** Using `sh -c "exec java $JAVA_OPTS -jar app.jar"` rather than the more common `exec-form` array lets JVM flags (heap sizing, for instance) be supplied at `docker run` time via an environment variable, without rebuilding the image — and `exec` ensures the JVM process replaces the shell as PID 1, so it still receives `SIGTERM` correctly for graceful shutdown.

10.2 Completing the Health Check: Adding Actuator

The Dockerfile's `HEALTHCHECK` and `docker-compose.yml`'s `service_healthy` condition both assume the application exposes `/actuator/health`. That assumption initially outran the implementation.

BUG: The Dockerfile referenced a health endpoint that didn't exist yet

`spring-boot-starter-actuator` was not originally on the classpath, so `/actuator/health` did not exist, and the health check — and by extension, anything in `docker-compose.yml` depending on the app container reporting healthy — had nothing real to check. The fix closed the gap end to end: the dependency was added; `application.yml` was configured to expose *only* health (not `env`, `beans`, or anything else Actuator can expose) with `show-details: never`, so an unauthenticated caller learns nothing beyond `UP/DOWN`; `SecurityConfig` was updated to permit `/actuator/health` specifically, since neither Docker's health check nor Compose's dependency check can supply a JWT; and the Dockerfile's check itself was upgraded from a bare TCP probe to the real HTTP check shown above.

10.3 Local Development: `docker-compose.yml`

The Compose file defines three services, one of which is optional.

```

services:
  postgres:

```

```

image: pgvector/pgvector:pg16 # NOT plain postgres - see Chapter 9
environment:
  POSTGRES_DB: ${DB_NAME:-pharmachain}
  POSTGRES_USER: ${DB_USER:-postgres}
  POSTGRES_PASSWORD: ${DB_PASSWORD:-postgres}
ports: ["${DB_PORT:-5432}:5432"]
volumes: ["pharmachain-data:/var/lib/postgresql/data"]
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-postgres} -d ${DB_NAME:-pharmachain}"]
  interval: 5s
  timeout: 5s
  retries: 10

app:
  build: { context: ., dockerfile: Dockerfile }
  depends_on:
    postgres: { condition: service_healthy }
  environment:
    DB_HOST: postgres
    JWT_SECRET: ${JWT_SECRET:-...}
    ANTHROPIC_API_KEY: ${ANTHROPIC_API_KEY:-}
    # Reaches an Ollama instance running on the HOST by default.
    OLLAMA_BASE_URL: ${OLLAMA_BASE_URL:-http://host.docker.internal:11434}
  extra_hosts: ["host.docker.internal:host-gateway"]
  ports: ["${APP_PORT:-8080}:8080"]

# Optional, off by default - opt in with '--profile with-ai'.
ollama:
  image: ollama/ollama:latest
  profiles: ["with-ai"]
  ports: ["${OLLAMA_PORT:-11434}:11434"]
  volumes: ["pharmachain-ollama:/root/.ollama"]

```

Listing 10.3: docker-compose.yml - condensed

DESIGN DECISION

Ollama is not containerized by default. The default assumption is that a developer already has Ollama installed and running on their host machine (a common local-AI-development setup), and `host.docker.internal` — resolved natively by Docker Desktop, and via the explicit `extra_hosts` entry on Linux — lets the containerized app reach it. Compose *profiles* provide an escape hatch for anyone who would rather containerize Ollama too: `docker compose -profile with-ai up` brings up a fourth, fully containerized Ollama service instead, at the cost of needing to override `OLLAMA_BASE_URL` to point at that container's service name rather than the host.

10.4 The CI/CD Pipeline

The GitHub Actions workflow has three jobs, chained by `needs:`, matching the conventional build → test → publish → deploy order.

```

build-and-test:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-java@v4
      with: { distribution: temurin, java-version: '21', cache: maven }

```



```

- run: mvn -B clean verify
- if: failure()
  uses: actions/upload-artifact@v4
  with: { name: surefire-reports, path: target/surefire-reports/ }

```

Listing 10.4: build-and-test: the only job that runs on every push and every pull request

WHY THIS MATTERS

GitHub-hosted `ubuntu-latest` runners ship a Docker daemon natively, so `BusinessRuleIntegrationTest`'s Testcontainers-based PostgreSQL container runs with no additional Docker-in-Docker configuration required. This detail matters more than it might appear: every sandboxed environment used to develop this project up to this point lacked both Docker and outbound access to Maven Central, meaning this CI job is the *first* place the full test suite — including every trigger-behavior assertion — can actually execute rather than being verified only by static analysis. Chapter 11 discusses this constraint and how the project compensated for it during development.

```

build-and-push-image:
  needs: build-and-test
  if: github.event_name == 'push' && github.ref == 'refs/heads/main'
  permissions: { contents: read, packages: write }
  steps:
    - uses: actions/checkout@v4
    - id: image
      run: echo "name=ghcr.io/${echo '${{ github.repository }}' | tr '[:upper:]' '[:lower:]'}" >> "$GITHUB_OUTPUT"
    - uses: docker/login-action@v3
      with: { registry: ghcr.io, username: ${{ github.actor }}, password: ${{ secrets.GITHUB_TOKEN }} }
    - uses: docker/setup-buildx-action@v3
    - uses: docker/build-push-action@v6
      with:
        push: true
        tags: |
          ${{ steps.image.outputs.name }}:latest
          ${{ steps.image.outputs.name }}:${{ github.sha }}
        cache-from: type=gha
        cache-to: type=gha,mode=max

```

Listing 10.5: build-and-push-image: only after tests pass, only on main

The image name is lower-cased explicitly because GHCR requires a lowercase repository path while `github.repository` preserves whatever case the GitHub username/org and repo name actually use. Every image is tagged twice: `:latest` for convenience, and the immutable commit SHA for traceability — a deployment can always be tied back to the exact commit it was built from.

```

deploy:
  needs: build-and-push-image
  if: ${{ needs.build-and-push-image.result == 'success'
    && secrets.RENDER_DEPLOY_HOOK_URL != '' }}
  steps:
    - run: curl -fsS -X POST "${{ secrets.RENDER_DEPLOY_HOOK_URL }}"

```

Listing 10.6: deploy: optional, no-ops unless a repository secret is configured

This job is deliberately inert by default. Render’s deploy hook is a single webhook URL from a service’s dashboard; without the corresponding repository secret configured, the `if:` condition evaluates to false and the job is skipped entirely, cleanly, with no configuration required to keep the rest of the pipeline working. Swapping Render for Railway or Fly.io means replacing only this final step with their equivalent CLI or API call — the build → test → publish ordering upstream of it is unaffected.

10.5 Summary

Table 10.1 lists every environment variable the containerized application reads, consolidated from the Dockerfile, `docker-compose.yml`, and `application.yml`.

Table 10.1: Runtime environment variables

Variable	Purpose
DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD	PostgreSQL connection
JWT_SECRET, JWT_EXPIRATION_MS	Token signing key and lifetime
ANTHROPIC_API_KEY	Chat model authentication
OLLAMA_BASE_URL	Embedding model endpoint
INGEST_COMPLIANCE_DOCS	Toggles startup RAG document ingestion

Chapter 11

Testing Strategy

11.1 Why an In-Memory Database Cannot Be Used Here

The conventional shortcut for testing a Spring Data JPA application — swap PostgreSQL for an in-memory database like H2 during tests, for speed and zero external dependencies — was considered and rejected outright for this project, for a reason specific to what makes PharmaChain interesting in the first place.

WHY THIS MATTERS

An in-memory substitute does not understand PL/pgSQL. It cannot execute `trg_deduct_stock_on_dispense`, `trg_prevent_bad_sales`, `trg_strict_batch_dates`, or `trg_audit_qc_changes`, because none of these are portable SQL — they are PostgreSQL-specific procedural code. A test suite running against H2 would still pass in full even if every trigger in Chapter 2 were silently deleted, because H2 was never exercising them to begin with. For a project whose entire thesis is that the database enforces real business rules, testing against a database that cannot enforce those rules would test nothing that actually matters.

The only database that can actually verify these guarantees is PostgreSQL itself, so the integration test suite runs against a real, disposable PostgreSQL instance via Testcontainers — a JUnit 5 extension that boots an actual Docker container for the duration of the test class.

11.2 Loading the Real Schema Correctly

Testcontainers' `PostgreSQLContainer` offers a convenience method, `withInitScript()`, for loading SQL at container startup. It was deliberately not used.

CAUGHT: A convenience method that would have corrupted the trigger definitions

`withInitScript()`'s statement splitter is a naive semicolon-based parser — it has no awareness of PostgreSQL's dollar-quoted (`$$ ... $$`) function bodies, which contain *their own* internal semicolons as part of ordinary PL/pgSQL statements (see every `CREATE OR REPLACE FUNCTION` in Chapter 2). Using it would have split each trigger function's body at the first internal semicolon, submitting a syntactically broken fragment as a "statement." The schema is instead loaded by copying the SQL files into the container and executing `psql -f` against them directly, exactly as a human would run them — `psql` understands dollar-quoting natively, because it uses the same parser the PostgreSQL server itself does.

```
private static final DockerImageName PGVECTOR_IMAGE =
    DockerImageName.parse("pgvector/pgvector:pg16")
        .asCompatibleSubstituteFor("postgres");

@Container
```

```

static PostgreSQLContainer<> POSTGRES = new PostgreSQLContainer<>(
    PGVECTOR_IMAGE)
    .withDatabaseName("pharmachain")
    .withUsername("postgres")
    .withPassword("postgres")
    .withCopyFileToContainer(
        MountableFile.forClasspathResource("db/01_schema_and_data.sql"),

        "/01_schema_and_data.sql")
    .withCopyFileToContainer(
        MountableFile.forClasspathResource("db/02_security_schema.sql"),

        "/02_security_schema.sql");

@DynamicPropertySource
static void datasourceProperties(DynamicPropertyRegistry registry) {
    registry.add("spring.datasource.url", POSTGRES::getJdbcUrl);
    registry.add("spring.datasource.username", POSTGRES::getUsername);
    registry.add("spring.datasource.password", POSTGRES::getPassword);
    // No Ollama here - skip startup RAG ingestion (Chapter 9), which would
    // otherwise try to call it and fail.
    registry.add("app.ai.ingest-compliance-docs-on-startup", () -> "false");
    // Guarantees AnthropicChatModel bean construction succeeds with no
    // real key present; nothing here calls the real Anthropic API.
    registry.add("spring.ai.anthropic.api-key", () -> "test-key-placeholder");
}

```

Listing 11.1: Container setup and schema loading

Note the `pgvector/pgvector:pg16` image, not plain `postgres:16` — the same substitution explained in Chapter 9, required here because the application context under test includes the `VectorStore` bean, which needs the `vector` extension to exist regardless of whether any AI feature is actually exercised in a given test.

11.3 An Honest Constraint on This Entire Project

WHY THIS MATTERS

Every phase of this project up to the point this guide was written was developed in a sandboxed environment with neither a Docker daemon nor outbound network access to Maven Central. Static verification — a hand-written tokenizer confirming brace/parenthesis balance (Chapter 12 discusses building this tool correctly), cross-checking every internal import against a declared type, validating XML/YAML well-formedness — was the best verification available at build time. The GitHub Actions `build-and-test` job (Chapter 10) is the first environment in this project's history where `mvn clean verify` can actually run end to end, including this entire integration test suite. This is stated plainly rather than glossed over, because a testing chapter that did not acknowledge the limitations of how its own tests were verified during authorship would be misleading about its own rigor.

11.4 Test Inventory

Sixteen test methods cover four categories: authentication, the four triggers from Chapter 2, role-based authorization, malformed-input handling, and — added specifically during the

architecture review — regression coverage for the duplicate-id defect described in full in Section 12.2.

Table 11.1: Every test method and what it proves

Test	Proves
<i>Authentication</i>	
loggingInWithValidCredentials ReturnsABearerToken	Login issues a well-formed token
loggingInWithAWrongPasswordIs Rejected	401, UNAUTHENTICATED
callingAProtectedEndpointWith NoTokenIsRejected	401 without a token
aSalesAccountCannotDelete MasterData	403 for the wrong role
<i>Triggers (Chapter 2)</i>	
dispensingMoreThanAvailable StockIsBlockedByTheDatabase Trigger	trg_deduct_stock_on_dispense
sellingABatchThatFailedQcIs BlockedByTheDatabaseTrigger	trg_prevent_bad_sales (FAILED case)
sellingAnUntestedBatchIs BlockedByTheDatabaseTrigger	trg_prevent_bad_sales (untested case)
sellingABatchThatPassedQc Succeeds	The positive case, end to end
creatingABatchWithAFuture ManufacturingDateIsRejected	trg_strict_batch_dates
<i>Regression tests added during the architecture review</i>	
creatingAMaterialWithAnIdThat AlreadyExistsIsRejected NotSilentlyOverwritten	Section 12.2 fix
dispensingTheSameLotToTheSame BatchTwiceIsRejected NotSilentlyMerged	Same, composite-key case
aNonNumericBatchNumberInThePath Returns400NotA500	Type-mismatch handler (Chapter 7)
malformedJsonBodyReturns400Not A500	Malformed-JSON handler
auditorCannotCreateMaterials	AUDITOR role restriction (Chapter 8)
warehouseManagerCanCreate Materials	The restriction isn't <i>too</i> strict

11.5 A Representative Trigger Test

```
/** Batch 5003's only QC report (PQC003) is FAILED - trg_prevent_bad_sales
    must block the sale. */
@Test
void sellingABatchThatFailedQcIsBlockedByTheDatabaseTrigger() throws Exception {
    mockMvc.perform(post("/api/v1/sales")
        .header("Authorization", "Bearer " + adminToken)
        .contentType(MediaType.APPLICATION_JSON)
        .content(""))
```

```

        {"invoiceNo": 9101, "transactionDate": "2024-01-15",
         "currency": "INR", "accountNo": "ACC011",
         "totalValue": 5000.00, "batchNo": 5003, "saleQty":
10}
        ""))
        .andExpect(status().isUnprocessableEntity())
        .andExpect(jsonPath("$.code").value("DB_RULE_VIOLATION"));
    }

```

Listing 11.2: Verifying `trg_prevent_bad_sales` end to end, through the real API

Every fixed value in this test — account `ACC011`, batch `5003`, its FAILED QC report `PQC003` — is drawn from the project’s own seed data, verified by directly grepping `db/01_schema_and_data.s` for each value before writing the assertion, rather than assumed to exist. Section 12.3.2 in Chapter 12 explains why this verification step mattered in practice: several new regression tests initially risked colliding with existing seed data before this check was applied systematically.

11.6 A Representative Regression Test

```

@Test
void creatingAMaterialWithAnIdThatAlreadyExistsIsRejectedNotSilentlyOverwritten
()
    throws Exception {
    String body = ""
        {"materialId": "MAT999", "materialName": "Test Material", ...}
        "";

    // First creation succeeds...
    mockMvc.perform(post("/api/v1/materials")
        .header("Authorization", "Bearer " + adminToken)
        .contentType(MediaType.APPLICATION_JSON)
        .content(body))
        .andExpect(status().isCreated());

    // ...a second creation with the same id must be rejected, not
    // silently accepted as an edit.
    mockMvc.perform(post("/api/v1/materials")
        .header("Authorization", "Bearer " + adminToken)
        .contentType(MediaType.APPLICATION_JSON)
        .content(body))
        .andExpect(status().isUnprocessableEntity())
        .andExpect(jsonPath("$.code").value("BUSINESS_RULE_VIOLATION"));
}

```

Listing 11.3: Proving the duplicate-id fix, not just asserting it exists

This test does not merely check that a duplicate submission fails — it first proves the *first* submission succeeds, so a future regression that broke material creation entirely could not masquerade as this test passing for the wrong reason.

Chapter 12

The Architecture Review

12.1 Methodology

After the platform reached feature completeness — REST and JPA, security, Spring AI, and DevOps, in that order — a distinct, formal review pass was conducted, adopting the explicit posture of a senior engineer auditing someone else’s finished work rather than continuing to build. The review had four concrete methods, not just a general re-read:

1. **Line-by-line reading of every security-critical file**, on the theory that authentication and authorization code is where a subtle mistake causes the most damage for the least visible symptom.
2. **Systematic cross-referencing of every CHECK constraint** in `db/01_schema_and_data.sql` against the corresponding application-level validation, rather than spot-checking a few fields and hoping the pattern held.
3. **Tracing Spring Data JPA’s actual runtime behavior** for every entity with a client-assigned identifier, which is what surfaced this chapter’s most significant finding.
4. **Re-verifying every static analysis tool used during development**, after one of them produced a false positive — Section 12.5 tells this story in full, because catching a flaw in your own verification process is exactly the kind of thing a review like this exists to do.

Sixteen concrete defects were found and corrected. This chapter documents the most significant one in full mechanical detail, summarizes the rest with pointers to where each was already covered in context, and is explicit about what was checked and found to be *already correct* — a review that only ever reports bad news is not a complete picture.

12.2 The Central Finding: `save ()` Silently Corrupts Data on a Duplicate Client-Supplied ID

12.2.1 The Mechanism

Spring Data JPA’s `SimpleJpaRepository.save(S entity)` makes a decision every time it is called, based on a single question: does this entity’s `@Id` field look like it belongs to a new row, or an existing one?

```
public <S extends T> S save(S entity) {
    if (entityInformation.isNew(entity)) {
        entityManager.persist(entity);    // INSERT - fails loudly on a
        duplicate id
        return entity;
    } else {
        return entityManager.merge(entity); // UPDATE-or-INSERT - never fails
        on a duplicate id
    }
}
```

Listing 12.1: The actual logic inside Spring Data JPA’s `save ()` (simplified)

For an entity with no `@Version` field and no `Persistable<ID>` implementation — which describes every entity in `PharmaChain` — Spring Data’s default `isNew()` check is simply *is the @Id field null*. Every identifier in this project that is generated by the database (`Warehouse.itemId`, `ProductionLog.logId`, and a handful of others using `GenerationType.IDENTITY`) is genuinely null before the first save, so this check works correctly for them. But every identifier that is *client-supplied* — a material’s code, a batch number, an invoice number, a QC report id, the composite key on a dispensing record — is always non-null by the time `save()` is called, because the caller had to supply it. For every one of these entities, `isNew()` always returns `false`, and `save()` always calls `merge()`, never `persist()`.

`entityManager.merge()` has no concept of “this id should not already exist.” Its entire purpose is to reconcile a detached object’s state with whatever row currently has that id in the database — if no such row exists yet, it behaves like an insert; if one already does, it overwrites that row’s columns with the new object’s values. **No exception is thrown either way.** A duplicate `materialId` submitted to `POST /api/v1/materials` a second time does not fail. It silently replaces the first material’s name, type, storage condition, and every other field with whatever the second request contained.

12.2.2 Why This Is Worse Than It Sounds for Composite Keys

The single most dangerous instance of this bug was not a master-data table — it was `Material_Dispensing`, whose composite key is `(Batch_No, Item_ID)`.

```
CREATE TRIGGER trg_deduct_stock_on_dispense
AFTER INSERT ON Material_Dispensing -- NOT "OR UPDATE"
FOR EACH ROW
EXECUTE FUNCTION automate_warehouse_stock();
```

Listing 12.2: `trg_deduct_stock_on_dispense`, restated - fires on INSERT only

Submitting the same `(batchNo, itemId)` pair to `POST /api/v1/batches/{batchNo}/dispense` a second time, before the fix, would not fail — it would `merge()`, producing an `UPDATE` statement against the existing `Material_Dispensing` row that silently changed `quantity_issued` to whatever the second request specified. Because the trigger is bound to `INSERT` only, **it would not fire on that UPDATE at all**. The recorded dispensed quantity would change; `Warehouse.stock` would not be adjusted to match. The stock ledger would drift from physical reality, silently, with no error anywhere in the system to indicate anything had gone wrong.

12.2.3 The Fix, and the Alternative That Was Rejected

Two fixes were considered.

DESIGN DECISION

Considered and rejected: `Persistable<ID>`. Implementing `Persistable<ID>` on every affected entity — with a transient `isNew` flag defaulting to `true` via `@Builder.Default`, and a `@PostLoad` callback flipping it to `false` once Hibernate has hydrated an entity from an existing row — would force `save()` to always call `persist()` for a freshly-built entity, which *does* throw on a genuine primary-key collision. This is the more architecturally elegant fix, and it repairs the root cause once, for every entity, rather than patching each call site. It was rejected specifically because of the environment this project was developed in: with no route to a Java compiler

validating each change (Chapter 11 explains this constraint), a change touching entity structure, Lombok's builder-default interaction, and a JPA lifecycle callback across roughly fifteen files was judged to carry meaningfully more risk-of-silent-breakage than a smaller, easier-to-verify-by-eye change — and getting the *fix* for a data-corruption bug wrong is a worse outcome than leaving the bug better documented and deferred. **Chosen: an explicit `existsById` guard, added at each call site.** Smaller, more repetitive, and far easier to confirm correct by reading each individual change in isolation.

```
/**
 * Spring Data JPA's save() calls merge() (not persist()) whenever the
 * entity's @Id is already non-null - which is always true here, since
 * materialId is client-supplied, not generated. merge() on an id that
 * already exists silently UPDATES that row instead of failing, so a
 * duplicate-id "create" would otherwise overwrite someone else's material
 * with no error at all. This check is what turns that into a clear 422
 * instead.
 */
@Transactional
public MaterialMaster create(MaterialMaster material) {
    if (repository.existsById(material.getMaterialId())) {
        throw new BusinessException(
            "Material '" + material.getMaterialId() + "' already exists");
    }
    return repository.save(material);
}
```

Listing 12.3: The fix, as applied to `MaterialService.create()` - one of thirteen identical patches

Table 12.1 lists every one of the thirteen methods this exact pattern was applied to.

WHY THIS MATTERS

Two entries in this table deserve a specific note. `PurchaseService` and `SalesService` both guard `Transaction.invoiceNo` rather than a table-specific key, because `Transactions` is a single shared ledger for both buy and sell operations (Chapter 2). A single `existsById` check in either service therefore also catches the case where a client accidentally reuses an invoice number that was already used for the *other* direction — a purchase invoice number resubmitted as a sale, or vice versa — which a narrower, table-scoped check would have missed.

Two regression tests specifically prove this fix (full listings in Chapter 11): `creatingAMaterialWithAnIdThatAlreadyExistsIsRejectedNotSilentlyOverwritten` and `dispensingTheSameLotToTheSameBatchTwiceIsRejectedNotSilentlyMerged`, the latter chosen specifically to exercise the more dangerous composite-key, trigger-bypass scenario.

12.3 Every Other Finding

The remaining fifteen findings were each explained in full mechanical detail in the chapter covering the subsystem they belong to. Two of them are worth reproducing in complete form here rather than only summarizing, because Chapters 5 and 11 each promised the full detail would appear in this chapter.

Table 12.1: Every method patched against the `merge()` vs. `persist()` defect

Method	Guard added
<code>MaterialService.create()</code>	<code>existsById(materialId)</code>
<code>AccountService.create()</code>	<code>existsById(accountNo)</code>
<code>EmployeeService.create()</code>	<code>existsById(empId)</code>
<code>EquipmentService.create()</code>	<code>existsById(equipmentId)</code>
<code>ProductService.create()</code>	<code>existsById(productId)</code>
<code>SupplierContractService.create()</code>	<code>existsById(contractId)</code>
<code>MaintenanceService.create()</code>	<code>existsById(maintenanceId)</code>
<code>BatchService.createBatch()</code>	<code>existsById(batchNo)</code>
<code>BatchService.dispenseMaterial()</code>	<code>existsById on the composite (batchNo, itemId) key</code>
<code>QualityCheckService.submitMaterialCheck()</code>	<code>existsById(reportId)</code>
<code>QualityCheckService.submitProductCheck()</code>	<code>existsById(reportId)</code>
<code>PurchaseService.recordPurchase()</code>	<code>existsById(invoiceNo) on the shared Transaction table</code>
<code>SalesService.recordSale()</code>	<code>existsById(invoiceNo) - same table, catches cross-purchase/sale collisions too</code>

12.3.1 The Complete Validation-Gap Audit

Chapter 5 previewed three of these; Table 12.2 is the complete result of cross-referencing every `CHECK` constraint in `db/01_schema_and_data.sql` against the application-level validation covering the same field.

Two fields with real `CHECK` constraints were deliberately left out of this table: `Formula_Master.Weight_per_tablet` and `Batch.Yield_Percentage` are both constrained in the database, but neither is writable through any current REST endpoint — there was no live application-level code path to add a matching constraint to. Both are listed instead under recommendations at the end of this chapter.

12.3.2 Verifying Test Data Against Real Seed Data, Not Assumptions

Every fixed value used in a regression test written during this review — a material id, a batch number, an invoice number — was checked against `db/01_schema_and_data.sql` directly before being used in an assertion, rather than assumed to be safe. This mattered concretely twice:

- The new `creatingABatchWithAFutureManufacturingDateIsRejected` test originally risked using a batch number that might collide with existing seed data. A targeted `grep` confirmed batch 7000 appears only as a substring inside larger numeric values (70000.00, 770000.00) in the seed file, not as an actual `Batch_No`, before it was used.
- The new duplicate-id regression tests (Section 12.2) needed genuinely unused identifiers — `MAT999`, `MAT998`, `MAT997` were each confirmed absent from the seed data with the same targeted search before being chosen, rather than picked arbitrarily and hoped to be safe.

Table 12.2: All thirteen validation gaps closed, matched against their exact database constraint

Field	Database constraint	Fix
MaterialMaster.shelfLife	CHECK (Shelf_Life > 0)	+ @Positive
MaterialMaster.reorderLevel	CHECK (Reorder_Level > 0)	@Positive
AccountMaster.accountType	CHECK (... IN ('Supplier', 'Distributor', 'Hospital'))	@Pattern
EquipmentMaster.status	CHECK (... IN ('Active', 'Maintenance'))	@NotBlank @Pattern
ProductMaster.salableOrSample	CHECK (... IN ('M', 'S'))	@NotBlank @Pattern
ProductMaster.genericOrBranded	CHECK (... IN ('G', 'B'))	@NotBlank @Pattern
Supplier	NOT NULL CHECK (> 0)	@NotNull
Contract.agreedPrice		@Positive
MaintenanceLog.cost	NOT NULL CHECK (>= 0)	@NotNull @PositiveOrZero
SubmitMaterialQcRequest.sampleSize	CHECK (Sample_Size > 0)	+ @Positive
SubmitProductQcRequest.sampleSize	CHECK (Sample_Size > 0)	+ @Positive
CreateBatchRequest.stockQty	CHECK (Stock_Qty >= 0)	@PositiveOrZero
RecordPurchaseRequest.utQA	VARCHAR(2)	@Size(max = 2)
ProductionLog start/end time	CONSTRAINT chk_time CHECK (End_Time > Start_Time)	@AssertTrue cross-field

WHY THIS MATTERS

A test that happens to pass because its fixture data accidentally avoided a collision with real seed data is not verified to be correct — it is verified to be lucky. Checking this explicitly, every time, is a small habit with an outsized effect on how much a passing test suite can actually be trusted.

12.4 Summary of All Findings

12.5 A Tooling Self-Correction, Told in Full

Worth documenting on its own, because it illustrates a failure mode in the review process itself, caught and corrected rather than shipped.

Early static verification of this project relied on a crude brace-balance check: `grep -o '{' file | wc -l` compared against the equivalent count of `}`. Applied to the Testcontainers integration test file, it reported a mismatch — 35 opening braces, 34 closing. The file was re-read line by line by hand and appeared completely correct.

The crude check was wrong, not the code. JSON fixtures inside Java text blocks (`"""{"username": "%s"}"""`) contain literal `{/}` characters as part of the JSON payload, which a naive character count cannot distinguish from actual Java code structure. A

Table 12.3: All sixteen findings from the architecture review

#	Finding	Severity	See
1	save() / merge() silent data corruption (13 call sites)	Critical	Section 12.2
2	JWT accepted for a disabled/deleted account	Critical	Section 8.4
3	Master-data writes had no role restriction	Critical	Section 8.7
4–16	Client-input 500s, 13 validation gaps vs. CHECK constraints	Important	Chapters 5, 7

more careful regex-based check — stripping string and text-block content before counting — was written next, and it reported the file as balanced, but then reported two *different* files (`JwtService.java`, `JwtAuthenticationFilter.java`) as broken instead. Those were also false positives, this time caused by a Javadoc comment containing a stray " character (inside the phrase `Authorization: Bearer <token>`), which confused a regex pipeline that stripped strings *before* stripping comments — the stray quote made the string-stripping regex swallow content across a comment boundary it should never have touched.

WHY THIS MATTERS

The actual fix was to stop chaining independent regexes against the same text and instead write a single-pass tokenizer: one linear scan through each file's characters, correctly treating `//` line comments, `/* */` block comments, `"..."` string literals, `'...'` character literals, and `"..."` text blocks as mutually exclusive, order-sensitive lexical contexts — exactly the way a real compiler's lexer would, rather than approximating it with independent substitutions. Every file in the project was re-verified with this corrected tool and found genuinely balanced. The broader lesson generalizes past LaTeX or Java specifically: a verification tool that has never been tested against a case designed to fool it has not actually been verified itself, and a review process that reports its own tooling mistakes has more credibility than one that reports only clean results throughout.

12.6 Verified Correct: What This Review Did *Not* Find Wrong

- **CORS plus credentials.** `setAllowedOriginPatterns(List.of("*"))`, not `setAllowedOrigins`, correctly reflects the actual request origin rather than a literal wildcard — the only combination browsers accept alongside `allowCredentials(true)`.
- **JWT secret length.** The default local-development HMAC secret is 66 bytes, comfortably over HS256's 32-byte minimum.
- **The token's role claim is never trusted for authorization** — every request re-reads the current role from the database.
- **AuthenticationManager wiring.** An explicit `DaoAuthenticationProvider` bean means Spring Boot does not also construct a redundant, competing default provider.
- **The generic "invalid username or password" login message** is intentional (Chapter 8), not an oversight — it avoids leaking account-existence or account-status information to an attacker.

- **No SQL injection surface anywhere.** Every raw-SQL code path (`DashboardService`, `RecallService`) uses parameterized `?` placeholders exclusively.
- **Every composite-key `@Embeddable` class** correctly implements `Serializable` and `@EqualsAndHashCode`, both required for JPA composite identifiers to function at all.

12.7 Recommendations, Deliberately Not Implemented

Flagged rather than treated as scope creep for this review:

- **Pagination.** Every list endpoint returns an unbounded `List`. Fine at seed-data scale; would need `Pageable` before this schema held production-scale row counts.
- **Login rate limiting.** No brute-force protection on `POST /api/v1/auth/login`. `Bucket4j` or equivalent would be the natural fit — net-new infrastructure, not a “fix.”
- **`Formula_Master` has no REST endpoint at all.** Not a bug — nothing calls it, so nothing is broken — but a real coverage gap if bill-of-materials management is ever needed.
- **`Batch.Yield_Percentage`** is similarly not writable through any current endpoint.

Chapter 13

Retrospective

13.1 What Actually Changed

Nothing in `db/01_schema_and_data.sql`'s triggers, views, or procedure was rewritten over the course of this project — the one edit made to that file at all was a single missing semicolon after its `set search_path` statement, which broke a standalone `psql -f` run and was fixed with a one-line, clearly-commented change, documented rather than silently patched. Everything else described across the preceding twelve chapters is additive: a REST and JPA layer that exposes the schema's existing guarantees, a security layer that gates who can invoke them, an AI layer that can explain and query them in natural language, a container and CI/CD layer that ships the whole thing, and a test suite that proves all of it against the real database rather than an approximation of one.

The guiding principle from Chapter 1 — *the database is not legacy; it is the source of truth for every business rule it already enforces* — held up under an adversarial re-reading of the finished platform. The sixteen defects documented in Chapter 12 were all found *above* the database, in the layers built to expose it: an ORM behavior nobody had traced through to its logical conclusion, a security check that was never added, error handling that hadn't yet accounted for every input a real client would eventually send. Not one of them was a flaw in the trigger logic itself, which is exactly what should be true of a system where the database was trusted as the last line of defense rather than worked around.

13.2 From Coursework to Something Closer to Production

Table 13.1 is a deliberately blunt before/after comparison.

Table 13.1: From a database coursework project to a reviewed, deployable platform

Dimension	Before	After
Interface	<code>psql</code> , direct SQL	Versioned REST API, OpenAPI docs
Access control	PostgreSQL roles, if any	JWT, 6-role RBAC, method security
Intelligence	None	RAG copilot, structured-output drafting
Deployment	Manual DDL load	Multi-stage Docker, Compose, CI/CD
Verification	Manual <code>psql</code> testing	Testcontainers integration suite
Defect process	Informal	Sixteen defects, formally reviewed and fixed

13.3 What Would Change With More Time

Chapter 12 already lists the specific recommendations deliberately left unimplemented — pagination, login rate limiting, and the two genuinely uncovered fields (`Formula_Master`, `Batch.Yield_Percentage`). Two broader directions are worth naming here as well, since they extend the project’s existing patterns rather than introduce new ones:

- **The `Persistable<ID>` fix, done properly, with a compiler available.** Section 12.2 explains in detail why the smaller, thirteen-call-site fix was preferred given this project’s development constraints. With a working `mvn compile` loop — which the CI pipeline from Chapter 10 now finally provides — the more architecturally correct fix, closing the defect at its root for every current and future entity at once, becomes a much lower-risk change worth revisiting.
- **Extending the compliance copilot’s tool set.** `DashboardAiTools` currently exposes exactly the three read-only views. A natural next step, following the same safety posture (named, single-purpose, read-only methods; no raw SQL access), would add tools over QC history and maintenance logs, broadening what the copilot can answer without widening what it can touch.

13.4 Closing

This guide documented a specific, opinionated way of extending a well-built database project into a full platform: preserve what the database already guarantees, add exactly the layers a production system needs on top, and — critically — go back afterward and check the finished work as skeptically as if someone else had built it. The sixteen defects in Chapter 12 are not a mark against the earlier chapters; they are what that final step is supposed to find, and finding them there, before a user did, is the entire point of doing it at all.

Chapter A

Reference Material

A.1 Complete API Reference

Every endpoint in PharmaChain, grouped by resource. Unless marked *public*, every endpoint requires `Authorization: Bearer <token>`; the **Access** column names the additional role restriction beyond “authenticated,” where one exists.

Table A.1: Authentication

Endpoint	Access	Description
POST /api/v1/auth/login	public	Exchanges credentials for a JWT
POST /api/v1/auth/register	ADMIN	Creates a new login account

Table A.2: Master data (plain CRUD - GET open to any role on every resource below)

Resource	Base path	Write access
Materials	/api/v1/materials	ADMIN, WAREHOUSE_MANAGER; delete: ADMIN
Accounts	/api/v1/accounts	ADMIN only
Employees	/api/v1/employees	ADMIN only
Equipment	/api/v1/equipment	ADMIN, WAREHOUSE_MANAGER, PRODUCTION_SUPERVISOR; delete: ADMIN
Products	/api/v1/products	ADMIN, PRODUCTION_SUPERVISOR; delete: ADMIN
Supplier contracts	/api/v1/supplier-contracts	ADMIN, WAREHOUSE_MANAGER; delete: ADMIN
Maintenance logs	/api/v1/maintenance-logs	ADMIN, PRODUCTION_SUPERVISOR, WAREHOUSE_MANAGER

A.2 Demo Accounts

A.3 Environment Setup Quick Reference

```
# 1. Postgres (pgvector-enabled image, required for the AI vector store)
docker compose up -d

# 2. Load the schema - two files, in order
psql -h localhost -U postgres -d pharmachain -f db/01_schema_and_data.sql
psql -h localhost -U postgres -d pharmachain -f db/02_security_schema.sql
```

Table A.3: Production workflow

Endpoint	Description	Write access
GET /api/v1/batches	List / fetch a batch	any role
GET .../{batchNo}		
POST /api/v1/batches	Create a batch (trg_strict_batch_dates)	AD, WM, PS
GET .../{batchNo}/dispensing	Dispensing history	any role
POST .../{batchNo}/dispense	Dispense material (trg_deduct_stock_on_dispense)	AD, WM, PS
GET/POST /api/v1/production-logs	Process-stage log entries	AD, PS
GET/POST .../quality-checks/materials	Incoming raw-material QC	AD, QC
GET/POST .../quality-checks/products	Finished-goods QC (trg_prevent_bad_sales_gate)	AD, QC
GET .../products/{reportId}/audit-trail	trg_audit_qc_changes history	any role

AD = ADMIN, WM = WAREHOUSE_MANAGER, PS = PRODUCTION_SUPERVISOR, QC = QC_ANALYST

```
# 3. AI features (optional - the rest of the API works without this)
ollama pull nomic-embed-text
export ANTHROPIC_API_KEY=sk-ant-...

# 4. Run
mvn spring-boot:run

# 5. Get a token
curl -X POST http://localhost:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{"username": "admin", "password": "Admin@123"}'

# 6. Run the full test suite (needs a local Docker daemon)
mvn clean verify
```

Listing A.1: From an empty checkout to a running, fully-featured local instance

A.4 Glossary

Table A.4: Commerce, AI, and dashboards

Endpoint	Description	Write access
POST /api/v1/purchases	Invoice + warehouse lot + line, atomically	ADMIN, WAREHOUSE_MANAGER
POST /api/v1/sales	Invoice + FG line (trg_prevent_bad_sales gate)	ADMIN, SALES
GET /api/v1/recalls	List / fetch recalls	any role
GET .../{recallId}		
POST /api/v1/recalls	Runs execute_product_recall	ADMIN, QC_ANALYST
POST /api/v1/ai/ask	RAG + tool-calling Q&A	any role
GET /api/v1/ai/recall-notice/{id}	Structured recall notice draft	ADMIN, QC_ANALYST
GET .../dashboards/inventory-shortage	v_inventory_shortage	any role
GET .../dashboards/expiry-risk	v_inventory_expiry_risk	any role
GET .../dashboards/traceability/{batchNo}	v_fda_batch_traceability	any role

Table A.5: Seeded accounts for local development and grading

Username	Password	Role
admin	Admin@123	ADMIN - unrestricted, including deletes and registration
qc.analyst	Qc@12345	QC_ANALYST - QC results, recalls
wh.manager	Wh@12345	WAREHOUSE_MANAGER - purchases, dispensing, batches
sales.rep	Sales@123	SALES - sales
auditor	Audit@123	AUDITOR - read-only across every write endpoint

Term	Meaning in this guide
Defense in depth	A rule enforced at more than one layer (service pre-check <i>and</i> database constraint), so the outer layer's failure never removes the guarantee
Trigger	PL/pgSQL procedural code bound to a table event (INSERT/UPDATE/DELETE), executed by PostgreSQL itself
persist() vs. merge()	JPA's two write operations; persist() fails on a duplicate id, merge() silently overwrites - the subject of Section 12.2
RAG	Retrieval-Augmented Generation - injecting retrieved reference content into a prompt before the model answers
Tool-calling	Giving a language model a fixed set of named, callable methods instead of open-ended query access
@PreAuthorize	Spring Security method-level authorization, implemented as an AOP proxy around the annotated method
Testcontainers	A JUnit extension that boots a real, disposable Docker container (here, PostgreSQL) for the duration of a test class